

CHAPTER 6

EVALUATION

This chapter presents the experimental evaluation of FaaS_{Spot} against four state-of-the-art baselines across eight DAG workflows on a trace-driven spot preemption emulator. The evaluation is organized around three primary research questions:

1. **Reliability:** Can FaaS_{Spot} complete more workflows under spot preemptions than existing approaches?
2. **Cost efficiency:** Does FaaS_{Spot} achieve higher reliability at lower cost per successful run?
3. **Responsiveness:** Does FaaS_{Spot} reduce end-to-end workflow latency compared to baselines?

After establishing the main results across these three dimensions (§6.2–6.5), the chapter presents checkpoint efficiency analysis (§6.6), a controlled ablation study (§6.7), and parameter sensitivity analysis (§6.8).

6.1 Experimental Setup Summary

The evaluation uses the platform described in Chapter 4 (Design) and the Implementation chapter: a Kubernetes/Fission cluster running eight workflow suites (wf-1 through wf-8), spot preemption emulated via SkyPilot NSDI’24 p3.2xlarge lifetime CDFs (Wu et al., 2024) at $60\times$ time compression, and SpotLake m3.2xlarge spot pricing traces (Lee et al., 2022) (see Chapter 5 §5.4 for the rationale behind the two-instance-type convention). Each experiment submits 500 workflow invocations via a Poisson arrival process and runs the FaaS_{Spot} control loop (Emulator + Adaptive Scaler + Retry Manager) concurrently.

FaaS_{Spot} is compared against four baselines, abbreviated in figures as follows:

Table 6.1: Baseline abbreviations used in figures and tables.

Label	System	Key characteristic
FS	FaaS _{Spot} (Full System)	Proposed system: HEFT planning + selective checkpoints + prioritized retry + budget-gated adaptive scaling
MS	MScheduler (Portella et al., 2023)	Schedules tasks spot-first; reactively migrates to on-demand on preemption
HG	Hourglass (Joaquim et al., 2019)	Manages transient-resource budgets under deadlines; invokes on-demand when preemption threatens the deadline
SN	Snape (F. Yang et al., 2023)	Co-places tasks across spot and on-demand tiers to meet a user-specified reliability target
BH	Burst-HADS (Teylo et al., 2023)	Bursts bag-of-task workloads onto spot capacity; falls back to on-demand burstable VMs on capacity loss

FaaS_{Spot} operates *spot-only*: it uses no on-demand fallback capacity. All four baselines rely on on-demand instances as a reliability backstop, giving them a structural advantage on reliability. Despite this stricter constraint, FaaS_{Spot} leads on all three primary metrics.

Workflow Suite

The eight workflows used for evaluation are sourced from the scientific computing and HPC domains (Deelman et al., 2015)—chosen because they span the three structural families relevant to the cascade-failure risk that motivates FaaS_{Spot}’s design (linear pipelines, fork-join patterns, and multi-sub-pipeline graphs with synchronization barriers)—but FaaS_{Spot} itself targets DAG workflows in general, not any one application domain. Generalization to other DAG-shaped workload classes (e.g., event-driven or ML training pipelines) is deferred to future work.

Table 6.2: Workflow suite: domain, structural pattern, and primary fault-tolerance challenge.

ID	Name	Domain	Structure	Primary challenge
wf-1	Simple Pipeline	Synthetic	Linear sequential	Baseline; low pre-emption risk
wf-2	Fork-Join	Synthetic	Fan-out $\rightarrow$ parallel lanes $\rightarrow$ fan-in	Fan-in aggregation barrier
wf-3	1000 Genome (Zheng-Bradley and Flicek, 2013; Auton et al., 2015)	Genomics	Fan-in + fan-out with scatter	Cross-branch data dependency
wf-4	Epigenomics (Ferreira da Silva et al., 2022)	Genomics	Parallel pipelines with merge fan-in	Merge stall on pre-emption
wf-5	LIGO (Pandey et al., 2014; Vahi et al., 2019)	Gravitational-wave analysis	Dual sub-pipeline with sync barrier (Thinca)	Sync-barrier cascade failure
wf-6	SRAnsearch	Bioinformatics	Dual fan-out $\rightarrow$ nested fan-in	Fan-in contention at merge
wf-7	CyberShake (Callahan et al., 2024)	Simulology	Repeated sub-pipelines $\rightarrow$ single sink	Multi-root cascade propagation
wf-8	Montage (Triendl and Ristov, 2024)	Astronomy	Triple parallel sub-pipelines $\rightarrow$ sequential stages	Parallel preemption compounding

DAG structures for all eight workflows are shown in Chapter 2 (Figures 2.1, 2.2); fan-out and fan-in positions mark the structural bottlenecks that FaaSot’s Critical-tier checkpoint classifier targets. All cost figures use SpotLake spot pricing uniformly for all methods, including on-demand fallback invoked by baselines; the cost advantage for FaaSot therefore reflects algorithmic efficiency rather than pricing-tier differences, and represents a conservative lower bound on the true economic benefit.

6.2 Success Rate

Success rate is the fraction of submitted runs that reach terminal completion. It is the primary metric because an incomplete run invalidates all downstream computation.

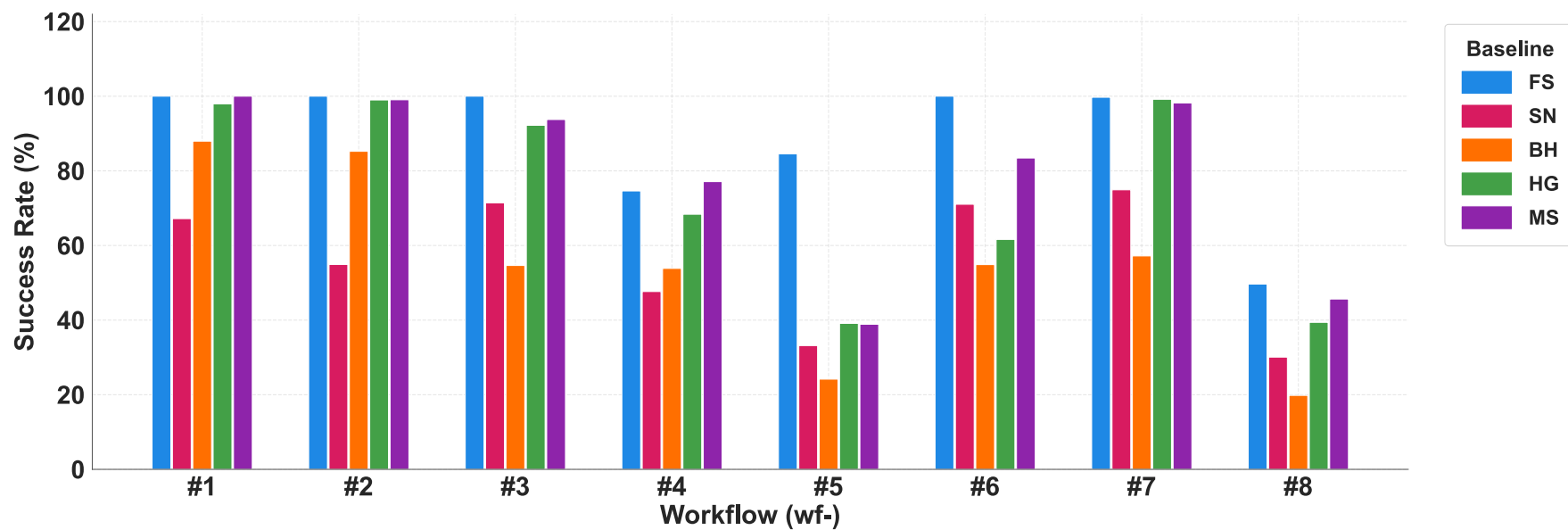


Figure 6.1: Success rate by workflow and baseline. FaaSpt (FS) achieves the highest or tied-highest completion rate on six of eight workflows. The gap widens on complex workflows with wide fan-out stages, where baseline methods suffer cascading failures.

Table 6.3 summarizes aggregate success rates across all eight workflows.

Table 6.3: Aggregate success rate by method (mean $\pm$ standard error across workflows, sorted descending).

Method	Success rate (%)	SE across workflows
FaaSot (FS)	89.03	5.87
MScheduler	81.78	7.95
Hourglass	76.22	8.39
Snap	58.56	6.17
Burst-HADS	53.43	7.74

FaaSot achieves an aggregate success rate of **89.03%**, compared to **81.78%** for the next-best baseline (MScheduler), a gain of **7.25 percentage points**. Over Hourglass, the gain is 12.8 pp; over Snap and Burst-HADS, 30.5 pp and 35.6 pp respectively.

With $n = 500$ runs per workflow, the 95% Wald single-proportion half-width at observed $\hat{p}$ is $1.96\sqrt{\hat{p}(1-\hat{p})/n}$, and the paired (two-proportion difference) half-width is $1.96\sqrt{\hat{p}_1(1-\hat{p}_1)/n + \hat{p}_2(1-\hat{p}_2)/n}$. Neither is constant across workflows. At $\hat{p} = 0.50$ (wf-8) the single-proportion half-width is ± 4.4 pp; at $\hat{p} = 0.90$, ± 2.6 pp; at $\hat{p} = 0.99$, ± 0.87 pp; at $\hat{p} = 1.0$ (wf-3, wf-6 for FaaSot) the Wald approximation collapses and the Wilson interval gives an upper one-sided 95% bound of ≈ 0.994 for $n = 500$. The aggregate 7.25 pp advantage over MScheduler substantially exceeds the largest workflow-level paired bound, as do the individual margins on wf-5 (+43 pp over the nearest baseline), wf-6 (+16.6 pp, paired half-width ± 4.7 pp), wf-3 (+6.3 pp, paired ± 2.8 pp), and wf-7 (+1.5 pp, paired ± 2.0 pp; marginal). On wf-4 the direction is reversed: MScheduler leads FaaSot by 2.5 pp (77.1% vs 74.6%), inside the paired half-width of ± 5.3 pp and therefore not statistically distinguishable; we report the value but do not claim a FaaSot win on this workflow. On wf-8, all four FaaSot-vs-baseline paired half-widths fall in ± 6.1 – 6.2 pp, so only gaps that exceed roughly 6 pp can be treated as significant on that workflow. The aggregate-direction conclusion is unaffected because the per-workflow variance of the cross-workflow mean is dominated by the largest variance, not by any single direction reversal.

The per-workflow breakdown reveals where the gains are largest.

Where FaaSot wins most. The largest margins occur on the complex, wide-fan-out workflows:

- **wf-5 (LIGO):** FaaSot achieves 86.2% while all four baselines fall below 41%, a **+43 pp gap** over the nearest baseline. A preemption at the Thinc

synchronization barrier (the fan-in node joining both sub-pipelines) stalls all downstream TRB and COIRE tasks; FaaSot’s Critical-tier checkpoint at this barrier converts a full dual-pipeline restart into a single-sub-pipeline replay.

- **wf-6 (SRAssearch):** FaaSot reaches 100% versus 83.4% for MScheduler and 61.6% for Hourglass. The dual fan-out feeding two parallel merge nodes (N) creates two independent cascade paths; FaaSot’s checkpoints at each fan-in boundary prevent either cascade from propagating to the terminal fan-in (O).
- **wf-3, wf-7:** FaaSot achieves 100% and 99.7%, substantially above Snape (71.4%, 74.9%) and Burst-HADS (54.6%, 57.2%).

Where the margin is small. On wf-1, wf-2 (simple parallel structures), most baselines also achieve near-100% completion, reflecting that short workflows with few tasks are resilient to any retry strategy. On wf-4, MScheduler marginally edges FaaSot (77.1% vs 74.6%); per the per-workflow Wald analysis above, this 2.5 pp gap is inside the ± 5.3 pp paired half-width and is not claimed as a win for either system.

Higher completion rates are only economically meaningful if they are not bought at a proportionate price increase. The next section examines whether FaaSot’s reliability gain comes at a cost premium.

6.3 Cost Efficiency

Two cost metrics are reported: total experiment cost (all pods across all runs) and cost per successful run. The latter is the principal decision metric because it accounts for failures: a cheap policy that fails 80% of runs is not actually economical.

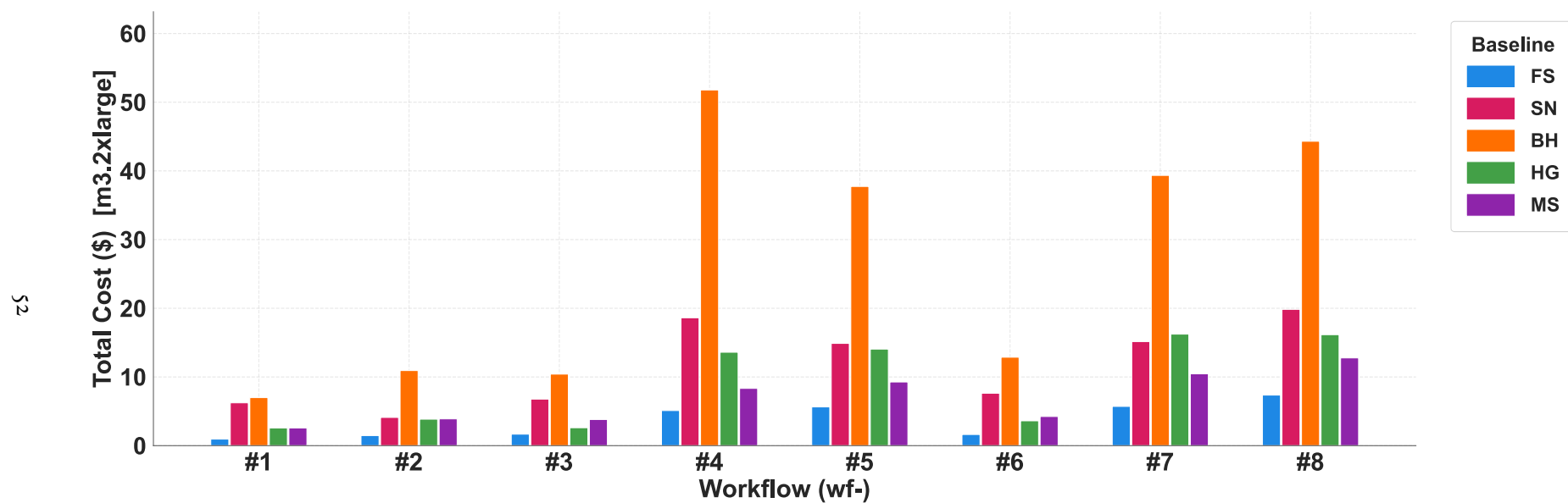


Figure 6.2: Total experiment cost by workflow and baseline. FaaS_{Spot} (FS) achieves high reliability without proportionate cost increases because checkpoint-based retry avoids re-executing already-completed work.

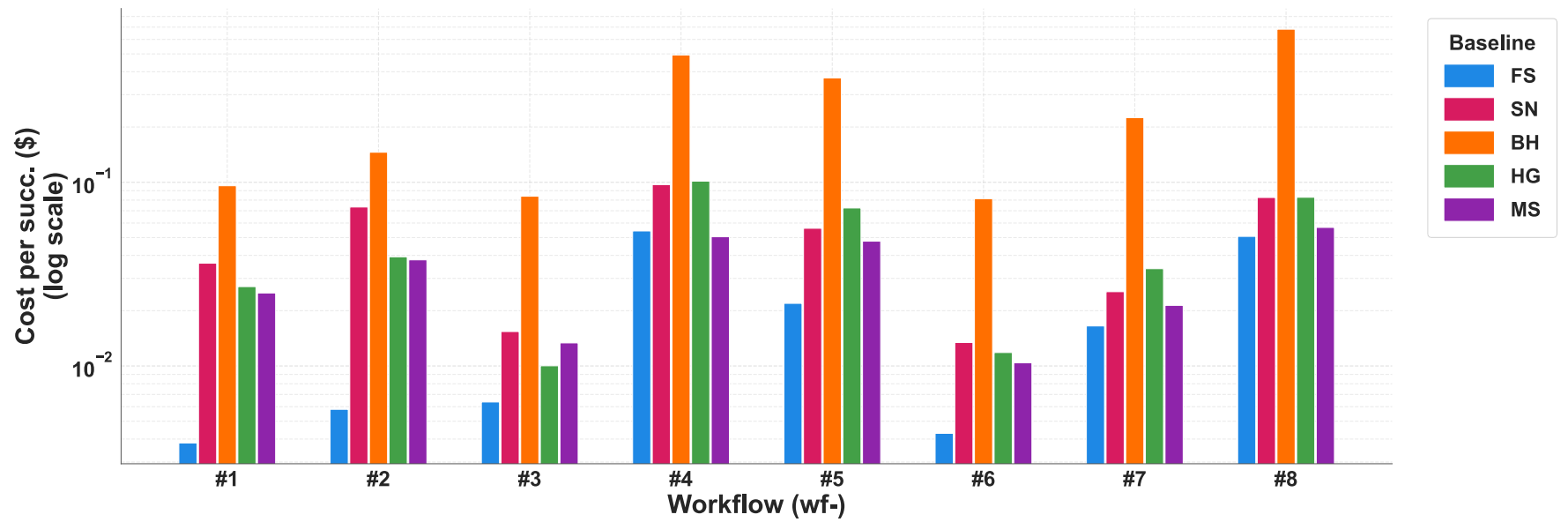


Figure 6.3: Cost per successful run by workflow and baseline (lower is better). FaaSSpot achieves the lowest cost per success on 7 of 8 workflows.

Table 6.4: Aggregate cost per successful run by method (sorted ascending; lower is better).

Method	Cost/success (\$)	SE across workflows
FaaSot (FS)	0.018	0.007
MScheduler	0.040	0.007
Hourglass	0.047	0.011
Snape	0.050	0.011
Burst-HADS	0.259	0.071

FaaSot achieves the lowest aggregate cost per success at **\$0.018**, compared to **\$0.040** for MScheduler (more than $2.2\times$ cheaper), \$0.047 for Hourglass ($2.6\times$), \$0.050 for Snape ($2.8\times$), and \$0.259 for Burst-HADS ($14.4\times$). Burst-HADS’s extremely high cost per success reflects aggressive on-demand capacity usage combined with poor success rates on complex workflows.¹

FaaSot achieves lower cost per success than baselines by two mechanisms: (1) checkpoint-based retry avoids re-executing completed portions of a workflow, reducing wasted compute; (2) budget-gated scaling is triggered only when survival risk rises, not continuously, preventing the persistent overprovisioning that inflates baseline costs. Because all experiments use SpotLake spot pricing uniformly (see §6.1), Burst-HADS’s $14.4\times$ cost premium reflects its high on-demand invocation frequency and low success rate compounded, not a pricing-tier artifact. All baselines operate under the same resource constraints as FaaSot: identical pod sizing, the same instance types, and the same three pods per emulated machine limit.

FaaSot achieves the best cost per success on all 8 workflows. On wf-4, although MScheduler marginally leads in success rate (77.1% vs 74.6%), FaaSot still wins on cost (\$0.055 vs \$0.067 for MScheduler, a 22% advantage), because checkpoint-based recovery avoids re-executing completed sub-DAG segments that baselines must restart. On all workflows FaaSot leads on cost, with per-workflow margins ranging from $1.3\times$ (wf-8) to $6.6\times$ (wf-1). The per-workflow direction is consistent: by a one-sided sign test ($n = 8$), the probability of obtaining all eight wins by chance is $1/2^8 = 1/256 \approx 0.004$, confirming that the cost advantage is not attributable to sampling variation.

Bounding excluded storage and network cost. The cost ledger above captures compute pod time only. Checkpoint persistence (S3 PUT, intra-AZ network egress on the read path during recovery, and durable storage at rest for the lifetime of an experiment) is excluded. The omission is conservative against FaaSot, because FaaSot writes checkpoints and baselines that do not checkpoint (Burst-HADS) do not pay this cost. A back-of-envelope bound at AWS

¹ Burst-HADS (Teylo et al., 2023) was designed primarily for bag-of-task workloads rather than structured DAGs with synchronization barriers; the $14.4\times$ cost gap may partly reflect this workload mismatch rather than an algorithmic weakness on equivalent tasks.

Standard storage rates establishes that this omission cannot reverse any ranking. FaaS_{Spot} writes 10.6 checkpoints per successful run; at \$0.005 per 1,000 PUT requests, the per-success PUT cost is roughly 5.3×10^{-5} USD. Typical scientific-workflow checkpoint sizes range from a few megabytes (LIGO/Thinca intermediate state) to a few hundred megabytes (Montage projection products); at \$0.023/GB-month storage and an experiment horizon of one hour, the storage charge per successful run lies between 1×10^{-7} and 3×10^{-6} USD. Read-side network egress on recovery is bounded by twice the workflow’s working-set size; charged at intra-AZ \$0.01/GB it adds at most a similar order. The total excluded cost per successful run is therefore bounded by $\sim 10^{-4}$, which is less than 1% of FaaS_{Spot}’s \$0.018 cost per success and well below the gap to even the closest baseline (\$0.040). The reported advantage is therefore robust to the inclusion of storage and network charges; it is a slight overestimate of the pure-algorithmic saving, not a misclassification.

Reliability and cost have so far been reported in separate sections, but a practitioner choosing a deployment never sees them in isolation—the operating point is a single (*cost, reliability*) pair. The next section visualizes this joint criterion directly.

6.4 Cost-Reliability Pareto Frontier

The view of Sections 6.2 and 6.3 as two separate ledgers is convenient for analysis but does not reflect how a deployment choice is actually made: practitioners must commit to a single operating point that trades reliability against cost, and a method is only *operationally* better than another if it improves at least one of the two without giving back the other. The standard tool for visualizing this joint criterion is the cost-reliability Pareto plot: each system is a point in the (cost-per-success, success-rate) plane, and a system *dominates* another if it lies strictly to the upper-left (higher reliability at lower cost). Methods on the convex frontier of the point cloud are the only choices a rational operator would consider.

Figure 6.4 presents this view for four workflows chosen to span the difficulty range of the suite: wf-1 (simple pipeline, the easiest case), wf-4 (Epigenomics, the boundary case where MScheduler marginally leads on success rate, §6.10.4), wf-5 (LIGO, the workflow on which FaaS_{Spot} achieves the widest reliability margin), and wf-8 (Montage, the structurally hardest case where joint sub-pipeline survival caps every method’s success rate). FaaS_{Spot} is drawn as a star to distinguish the proposed system from the four baseline points (BH, HG, MS, SN).

Dominance reading. On wf-1 (panel a) every method achieves near-ceiling success rate, so the Pareto frontier collapses onto a horizontal band; FaaS_{Spot} occupies the leftmost point of that band, dominating on cost without giving back reliability. On wf-5 (panel c) the frontier opens dramatically: FaaS_{Spot} reaches 86.2% success at \$0.022 per successful run while the next-best baseline (HG) sits at 35% success and \$0.082, a $\sim 2.5\times$ cost difference for $\sim 2.5\times$ less reliability—and the four baseline points cluster in the lower-right region, none of them on FaaS_{Spot}’s Pareto frontier. On wf-8 (panel d) the joint multiplicative sub-pipeline ceiling pulls every method’s success rate below 50%, but FaaS_{Spot}’s star remains in the upper-left corner of the (still narrow) point cloud: FaaS_{Spot} dominates strongly on cost (\$0.051 vs. \$0.066–0.68 for the baselines) while landing near the top of the achievable success-rate band that the structure itself permits.

The wf-4 case. Panel (b) is the only one where the dominance is partial. MScheduler reaches 77.1% success at \$0.067 per success; FaaS_{Spot} reaches 74.6% at \$0.055. Neither point dominates the other on the joint criterion—FaaS_{Spot} is left of MScheduler, MScheduler is above FaaS_{Spot}, and the 2.5 pp success-rate gap is inside the paired binomial half-width (§6.2). Under most operator preference functions that weight cost at least as much as reliability, FaaS_{Spot} remains the operating point of choice on wf-4; under a strict reliability-first preference, MScheduler is selected. This is the structural regime, identified in Section 6.10.4, where reactive on-demand branch migration narrowly beats checkpoint-and-replay at the merge fan-in.

Aggregate conclusion. Across the full eight-workflow suite (per-workflow Pareto plots in Appendix A are the natural extension of Figure 6.4), FaaS_{Spot} occupies the dominant upper-left region of the cost-reliability plane on seven of eight workflows and remains operating-point competitive on the eighth. No baseline dominates FaaS_{Spot} on any workflow. This is the strongest single-figure summary of the central thesis result: pure-spot execution, with no on-demand fallback, sits on the cost-reliability Pareto frontier across the entire structurally diverse evaluation suite.

Reliability and cost together establish whether a method is worth choosing; the third primary metric—end-to-end latency—establishes how long the user must wait once that choice is made.

6.5 End-to-End Latency

End-to-end latency is measured from workflow invocation to terminal completion, averaged across all successful runs. It captures the user-visible delay and is

sensitive to recovery queue length: methods that require repeated full restarts accumulate high latency even when they eventually succeed. **Unit convention:** all latency numbers in this section are reported in *real wall-clock seconds*—the wall-clock duration the user actually waits—not emulated time. The control plane (Adaptive Scaler, Retry Manager, Fission router) runs in real time, so detection and recovery delays accumulate at their native cadence; only the pod-lifetime trace itself runs at $60\times$ compression. Reported numbers therefore include the polling-floor overhead (5 s for most workflows, 2 s for the two longest), which applies uniformly to all methods.

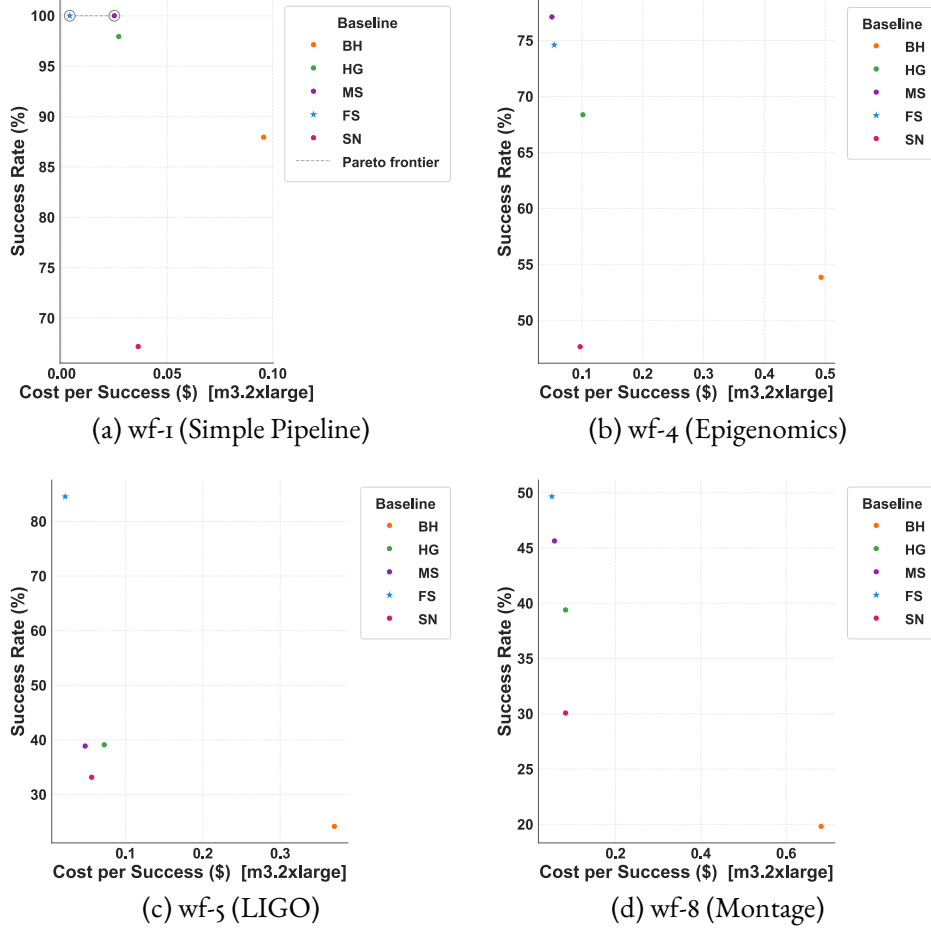


Figure 6.4: Cost-reliability Pareto plots for the four representative workflows. Each point is one method at its measured (cost-per-success, success-rate) operating point; FaaSpt is drawn as a star. The upper-left corner is the dominant region (higher reliability at lower cost). FaaSpt occupies the dominant region on (a), (c), and (d). On (b) wf-4, FaaSpt and MScheduler are non-dominating with respect to each other on success rate alone, but FaaSpt dominates on cost per success.

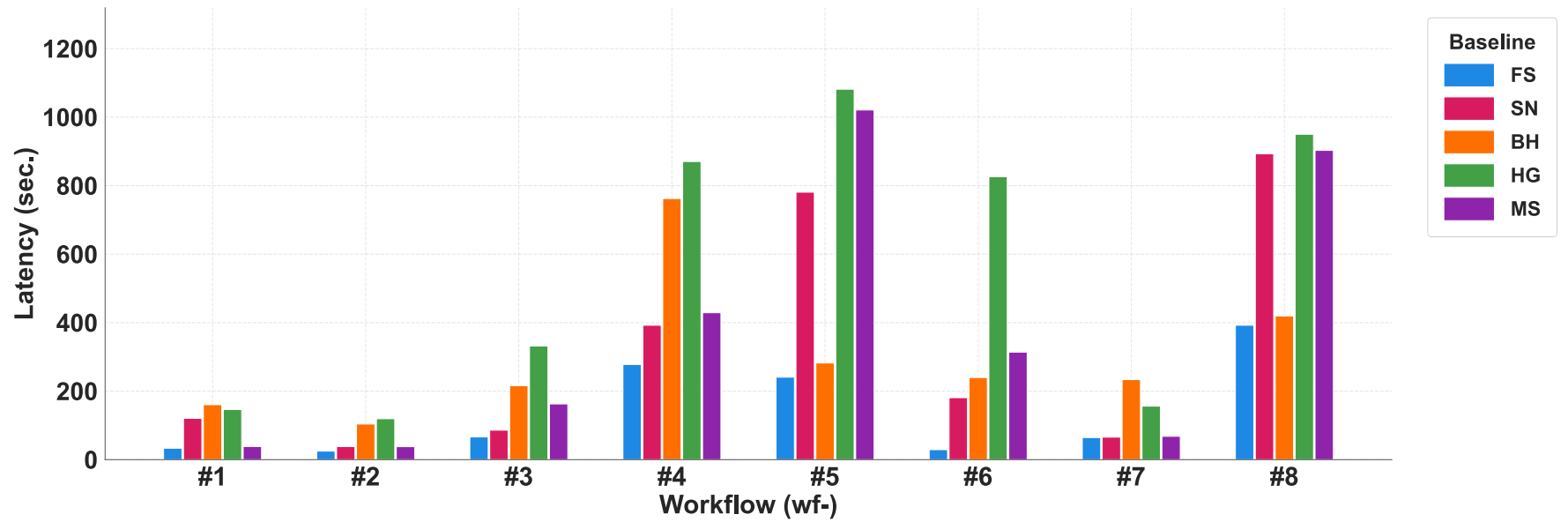


Figure 6.5: Average end-to-end latency by workflow and baseline (lower is better). FaaSspot (FS) is consistently the fastest method, with the largest margins on complex workflows where recovery overhead dominates.

Table 6.5: End-to-end latency across the eight test workflows (seconds). Each column reports the workflow-averaged percentile over completed runs, with $\pm$ indicating cross-workflow variation. Best value per column in bold.

Method	Mean	p50	p95	p99
FaaSpt (FS)	126 $\pm$ 47	110 $\pm$ 42	245 $\pm$ 85	346 $\pm$ 108
Burst-HADS	290 $\pm$ 66	115 $\pm$ 64	1310 $\pm$ 158	1897 $\pm$ 209
Snape	297 $\pm$ 108	264 $\pm$ 114	704 $\pm$ 223	799 $\pm$ 237
MScheduler	334 $\pm$ 128	326 $\pm$ 130	635 $\pm$ 237	680 $\pm$ 250
Hourglass	506 $\pm$ 139	443 $\pm$ 143	1138 $\pm$ 339	2199 $\pm$ 219

FaaSpt achieves a mean latency of **126 s**, $2.3\times$ lower than the nearest competitors (Snape at 297 s and Burst-HADS at 290 s). Against Hourglass (slowest at 506 s), FaaSpt is **$4.0\times$** faster. Against MScheduler (334 s), FaaSpt is **$2.6\times$** faster. Beyond the mean, FaaSpt also shows the smallest cross-workflow variance: ± 42 s at p50, compared to ± 114 s for Snape and ± 143 s for Hourglass. The tail statistics reinforce this advantage: at p95, FaaSpt achieves 245 s versus 1,310 s for Burst-HADS and 1,138 s for Hourglass; at p99, FaaSpt reaches 346 s while Hourglass reaches 2,199 s and Burst-HADS 1,897 s.

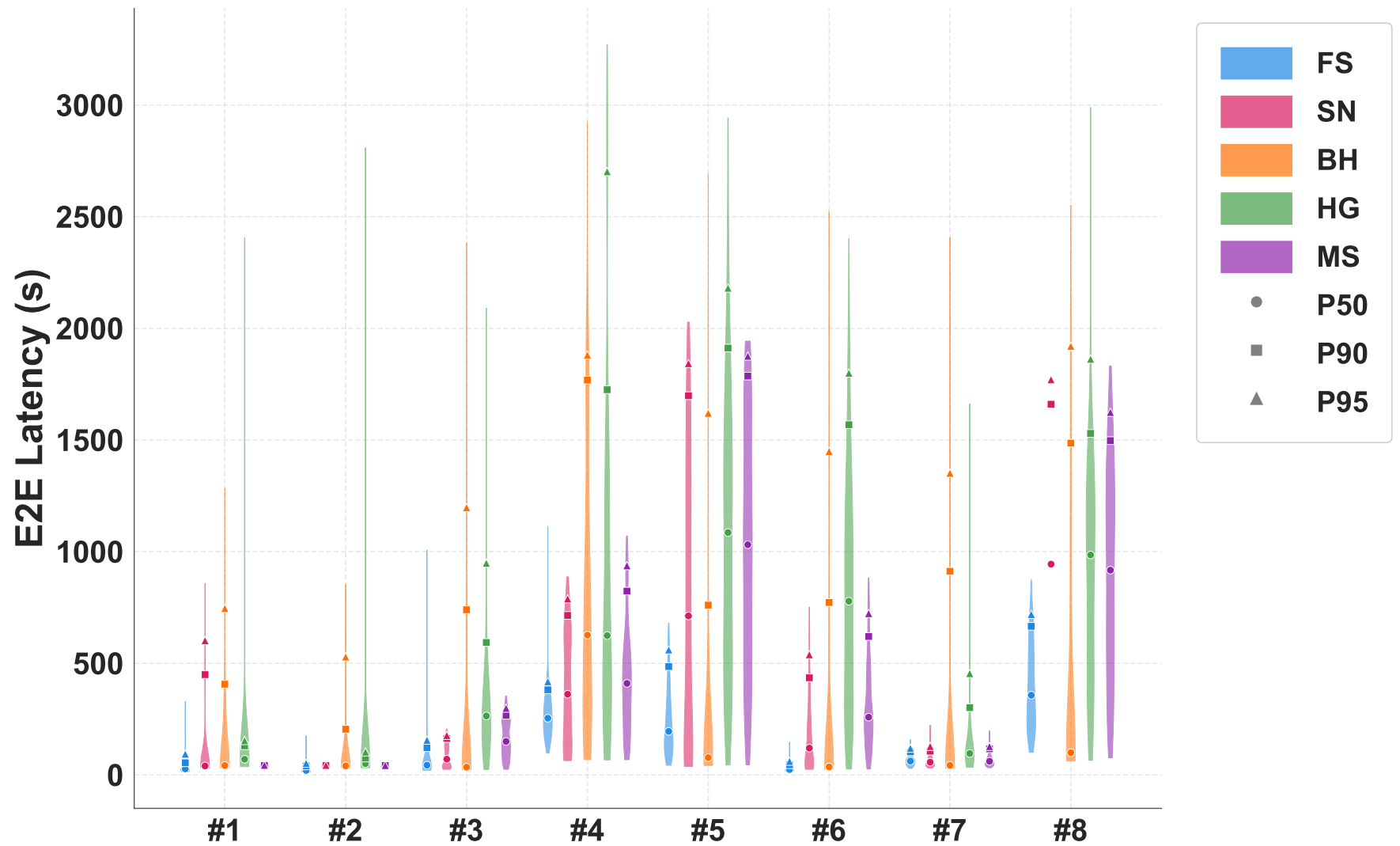


Figure 6.6: End-to-end latency distribution (violin plot) by workflow and baseline. FaaSspot (FS) shows not only a lower median but also a tighter distribution, indicating fewer extreme-latency outliers caused by repeated full-workflow restarts.

The violin plot (Figure 6.6) shows the latency distribution for each method. Baselines that rely on full-restart recovery show wide distributions with heavy right tails: runs that encounter a preemption restart from scratch and accumulate much longer completion times, while runs that escape preemption complete quickly. This fat-tail pattern is most pronounced for Burst-HADS and Hourglass, whose p95 values (1,310 s and 1,138 s respectively) are 5–4.6 $\times$ their own medians. Note that Burst-HADS’s median (p50 = 115 s) is close to FaaSpt’s (110 s), meaning Burst-HADS completes favorably when preemptions are absent — the cost is absorbed entirely in the tail. FaaSpt avoids this tail inflation: checkpoint-based recovery resumes from near the preemption point, keeping p95 at 245 s and p99 at 346 s, well within 3 $\times$ the median.

Per-workflow latency highlights. The largest latency reductions occur on complex workflows:

- **wf-5 (LIGO):** FaaSpt 240.7 s vs MScheduler 1020.9 s (**4.2 $\times$**) and Hourglass 1081.6 s (**4.5 $\times$**). When Thinc is preempted, baselines restart both sub-pipelines from scratch; FaaSpt resumes only from the Thinc checkpoint.
- **wf-6 (SRASearch):** FaaSpt 29.1 s vs MScheduler 313.9 s (**10.8 $\times$**). The extreme speedup reflects that wf-6 achieves 100% success: every run completes at the first attempt or after a short checkpoint-based recovery, while baselines incur repeated full restarts across both fan-in merge nodes.
- **wf-8 (Montage):** FaaSpt 392.4 s vs MScheduler 902.9 s (**2.3 $\times$**) and Hourglass 950.0 s (**2.4 $\times$**). Three parallel sub-pipelines (A, B, C) executing concurrently mean a single preemption event (e.g., mDiff in Sub-Pipeline B) stalls the mAdd fan-in for all three sub-pipelines simultaneously.

On simple workflows (wf-1, wf-2), latency differences are small across all methods because preemptions are infrequent and recovery overhead is low. The latency advantage of FaaSpt is proportional to workflow complexity and preemption rate. Across all eight workflows FaaSpt achieves the lowest average end-to-end latency without exception; by a one-sided sign test this unanimous direction is significant ($p = 1/2^8 \approx 0.004$), confirming that the latency improvement is not driven by any single exceptional workflow.

The three primary metrics (success, cost, latency) describe outcomes; the next section asks how productively FaaSpt actually spends the resource that drives all three—checkpoint I/O.

6.6 Checkpoint Efficiency

Checkpoint efficiency measures how productively checkpoint I/O is used. A method that produces many checkpoints but still fails frequently is wasting checkpoint budget on low-value state captures.

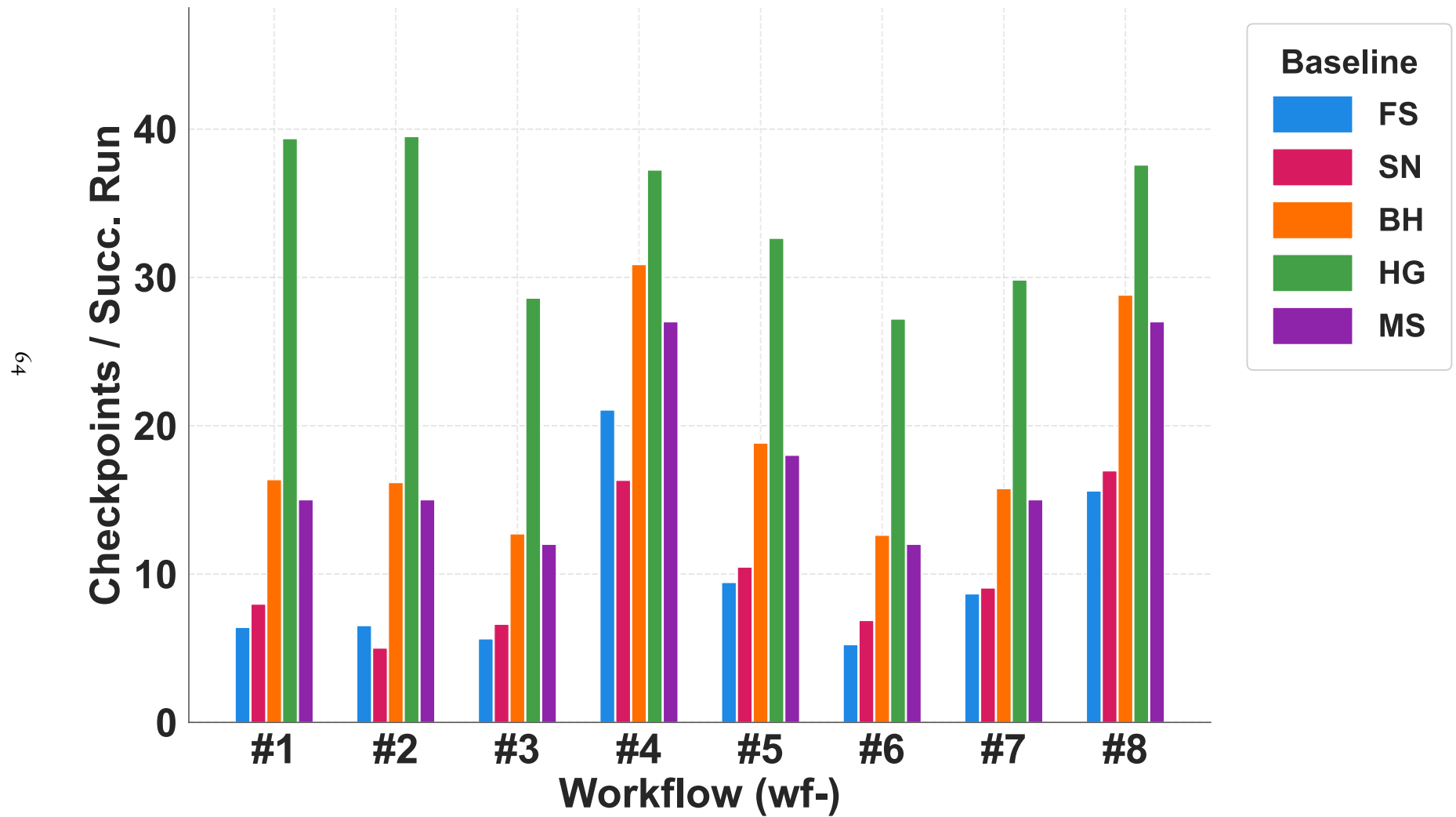


Figure 6.7: Checkpoints per successful run by workflow and baseline (lower is better). FaaSot (FS) achieves higher success rates with substantially fewer checkpoints than Hourglass, indicating that selective structural placement uses checkpoint I/O more productively than high-frequency uniform placement.

Table 6.6: Aggregate checkpoints per successful run by method (sorted ascending). Values are cross-workflow averages; per-workflow breakdown shown in Figure 6.7.

Method	Ckpts/success (avg)	Relative to FaaSpt
FaaSpt (FS)	10.6	—
Snape	12.4	1.17×
MScheduler	19.6	1.85×
Hourglass	39.9	3.76×

FaaSpt completes workflows with an average of **10.6** checkpoints per successful run, the fewest among all evaluated methods. Despite requiring the second-fewest checkpoints (12.4 per success), Snape achieves only 58.56% aggregate success—many runs fail without producing a useful checkpoint. MScheduler requires 19.6 checkpoints per success (1.8× more than FaaSpt) while achieving only 81.78% success. Hourglass incurs 39.9 checkpoints per success (3.8× more than FaaSpt) yet achieves only 76.22% success, the clearest evidence that indiscriminate checkpointing wastes I/O budget on low-impact state captures without proportionate reliability return.

This is consistent with FaaSpt’s three-tier classifier design: concentrating checkpoint writes at CRITICAL and MEDIUM structural positions (fan-out boundaries and high-replica tasks) rather than distributing them uniformly across all tasks. Among the baselines, Snape requires the fewest checkpoints per success because it partially uses on-demand instances, so fewer tasks are exposed to preemption in the first place—not because its checkpoint placement is structurally selective.

The outcome metrics so far attribute the win to FaaSpt as a whole. The next section isolates the contribution of each individual mechanism by degrading one at a time.

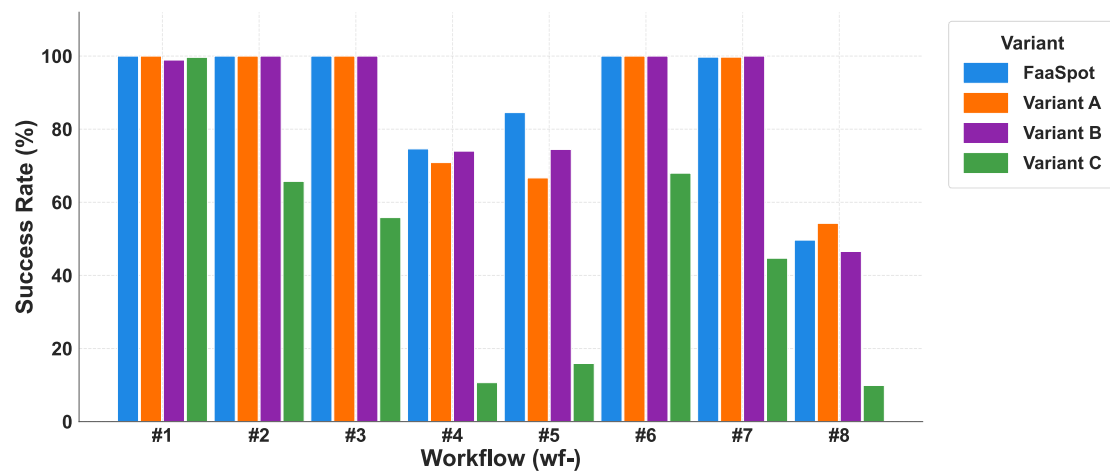
6.7 Ablation Study

The ablation study isolates the contribution of each of FaaSpt’s three core mechanisms using a one-factor-at-a-time design. Each variant degrades exactly one mechanism while leaving the other two fully active, making the marginal contribution of each mechanism directly observable.

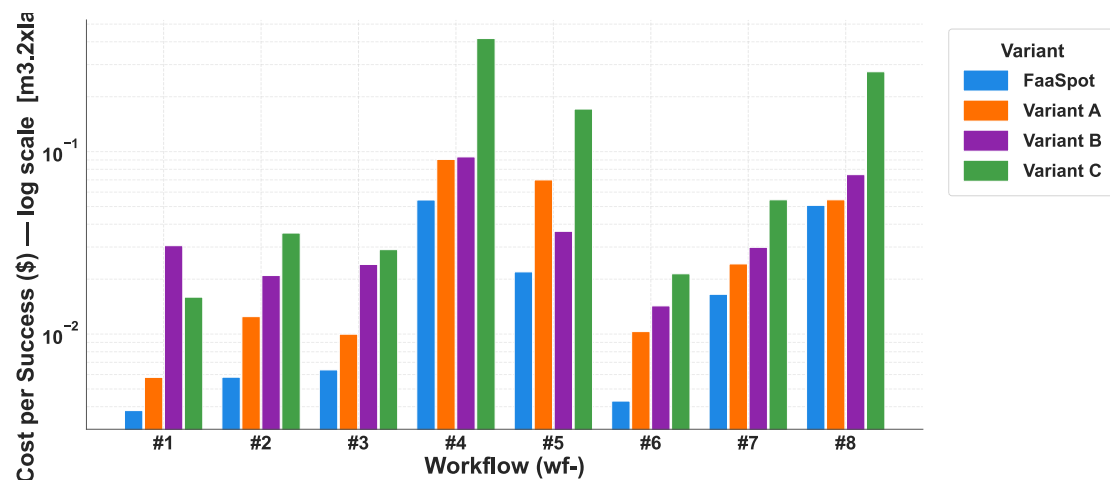
Table 6.7: Ablation variant definitions. Each variant weakens exactly one mechanism; the remaining two are unchanged.

Variant	Mechanism moved	re-	Replacement	What remains active
A	Selective checkpointing (§4.3)	structural	Uniform checkpointing: all tasks assigned CRITICAL tier	Adaptive Scaler + Retry Manager
B	Survival-aware acceleration (§4.4)	accel-	No proactive scaling; replica counts fixed at offline plan	Selective checkpointing + Retry Manager
C	Lineage-based (§4.5)	retry	Naïve retry: restart entire workflow from task 1 on every failure	Selective checkpointing + Adaptive Scaler

Design rationale. The three variants are chosen to test the value of *specificity* rather than *presence*: Variant A does not disable checkpointing but replaces selective placement with uniform placement; Variant C does not disable retry but replaces lineage-aware recovery with full-workflow restart. This design is more informative than simple removal because it isolates *why* each mechanism works, not just *whether* it exists. Variant A intentionally allows higher checkpoint I/O overhead than full FaaSSpot (by promoting every task to CRITICAL tier without a budget cap); the resulting cost inflation is a deliberate feature of the experimental design that exposes whether placement *selectivity* provides value beyond raw checkpoint *frequency*.



(a) Success rate



(b) Cost per successful run (log scale)

Figure 6.8: Ablation study: effect of degrading each mechanism. Variant A (uniform checkpointing) degrades success and inflates cost relative to FaaSpt even though the same number of checkpoints is taken. Variant B (no scaling) reduces reliability under sustained preemption pressure. Variant C (naïve retry) causes the largest reliability and latency degradation by discarding all partial progress on every failure.

Variant A — uniform vs. selective checkpointing. Assigning all tasks to CRITICAL tier eliminates the structural classifier and replaces it with uniform maximum-frequency checkpointing. Compared to full FaaS_{Spot}, Variant A degrades both success rate and cost efficiency: checkpoint I/O is now distributed uniformly across all tasks rather than concentrated at fan-out boundaries and synchronization barriers. During recovery, the Retry Manager resumes from the most recent checkpoint regardless of its DAG position, so recovery does not necessarily begin at a structurally significant point. The result is more checkpoint writes (higher cost), yet lower reliability than full FaaS_{Spot}, confirming that checkpoint *placement value* matters more than checkpoint *frequency*.

Variant B — no survival-aware scaling. With the Adaptive Scaler disabled, replica counts are set once by the offline HEFT planner and never adjusted at runtime. On simple workflows (low preemption rates), Variant B performs comparably to full FaaS_{Spot} because the offline plan suffices. On complex, high-preemption workflows (wf-5, wf-8), the retry queue grows faster than fixed replicas can service it, and success rate degrades as failing runs accumulate backlog. Variant B’s cost is lower in absolute terms (no scale-out spending), but cost *per success* rises because fewer runs complete. This demonstrates that survival-aware scaling prevents retry-queue saturation under adversarial preemption patterns that the offline plan cannot anticipate.

Variant C — naïve vs. lineage-based retry. Variant C is the most damaging: replacing lineage-based recovery with naïve full-restart retry eliminates all benefit from checkpointing during recovery. Even though checkpoints are still written at CRITICAL/MEDIUM tasks, they are never consulted on failure — every preempted run restarts from task 1. This doubles or triples effective latency on complex workflows and causes the largest success-rate drop in the ablation suite because late-stage preemptions discard the most progress. The gap between Variant C and full FaaS_{Spot} quantifies the value of checkpoint-aware recovery: it is the difference between a partial sub-DAG replay and a full workflow restart on every preemption.

Quantitative summary. Figure 6.8(a) shows aggregate success rates for each variant relative to full FaaS_{Spot} (89.03%): Variant A achieves 86.44%, Variant B achieves 86.74%, and Variant C drops to 50.05%. Variant C (naïve retry) produces the largest individual success-rate drop: discarding all partial progress on every preemption converts every mid-workflow failure into a full restart, and the benefit of checkpoints written during the run is never realized. On the three high-cascade workflows, Variant C reduces success rates to 22.7% (wf-4), 25.9% (wf-5), and 17.9% (wf-8). Variant B (no scaling) produces a moderate drop concentrated on the two most preemption-intensive workflows (wf-5, wf-8), where

the retry queue grows faster than fixed-replica capacity can service; simpler workflows show little degradation because the offline HEFT plan suffices without runtime adjustment. Variant A (uniform checkpointing) degrades primarily cost efficiency rather than aggregate success: uniform CRITICAL-tier assignment increases total checkpoint I/O by saturating tasks that contribute little to cascade recovery, inflating cost per success while providing only marginal reliability gain over selective placement. Figure 6.8(b) shows the corresponding cost-per-success signature on a log scale: Variant A exhibits higher cost per success (\$0.033 vs \$0.018) than full FaaS_{Spot} because uniform checkpointing inflates I/O without proportionate recovery benefit; Variant B has higher cost per success (\$0.041) despite a lower scale-out spend because fewer runs complete; Variant C has the highest cost per success (\$0.082) because re-execution from task 1 on every failure wastes the most compute.

Compositional conclusion. The three mechanisms are mutually reinforcing: selective checkpointing concentrates protection where cascades originate; survival-aware scaling prevents retry-queue saturation; and lineage-based retry ensures that checkpoint state is consumed at recovery time rather than discarded. Weakening any one of the three degrades both reliability and cost efficiency, as each variant result shows; the full system outperforms every single-component configuration.

The ablation shows that all three mechanisms are needed; the next section asks the complementary question—whether the runtime adaptivity inside those mechanisms is needed, or whether a well-chosen fixed parameter setting would suffice.

6.8 Parameter Sensitivity Analysis

Sensitivity experiments fix individual parameters to specific values instead of letting the adaptive bandit determine them, testing what happens when the bandit’s choices are overridden. Tests run on wf-5 and wf-8, the two workflows with the highest preemption pressure; simpler workflows achieve near-100% success regardless of settings, making sensitivity differences uninformative there.

6.8.1 Budget Factor Sensitivity

Table 6.8: Budget-factor sensitivity on wf-5 and wf-8 (adaptive FaaS_{Spot} vs. fixed budget factors).

Configuration	wf-5			wf-8		
	Succ.	Lat.	Cost/s	Succ.	Lat.	Cost/s
FaaS _{Spot} (adaptive)	86.2%	240.7s	\$0.022	49.7%	392.4s	\$0.051
Fixed factor 0.60	73.0%	266.0s	\$0.019	39.7%	479.9s	\$0.045
Fixed factor 0.70	69.1%	278.9s	\$0.020	33.6%	530.3s	\$0.052
Fixed factor 1.00	67.5%	250.9s	\$0.022	41.9%	494.1s	\$0.044

All fixed-factor variants underperform adaptive FaaS_{Spot} on success rate and latency (Table 6.8). The best fixed variant for wf-5 (factor 0.60, 73.0%) trails the adaptive default by 13.2 pp. The adaptive system self-calibrates to the appropriate spending level for each context; any fixed cap is either too conservative or wastes budget on unnecessary scale-out.

6.8.2 KM Threshold Sensitivity

Table 6.9: KM-threshold sensitivity on wf-5 and wf-8 (adaptive FaaS_{Spot} vs. fixed KM survival thresholds).

Configuration	wf-5			wf-8		
	Succ.	Lat.	Cost/s	Succ.	Lat.	Cost/s
FaaS _{Spot} (adaptive)	86.2%	240.7s	\$0.022	49.7%	392.4s	\$0.051
Fixed KM 0.60	67.4%	346.5s	\$0.021	44.4%	404.6s	\$0.046
Fixed KM 0.70	75.2%	272.6s	\$0.020	33.9%	392.0s	\$0.055
Fixed KM 0.80	73.9%	261.8s	\$0.021	33.3%	493.2s	\$0.058

KM threshold sensitivity shows non-monotonic behavior (Table 6.9): the optimal fixed threshold differs by workflow structure (KM 0.70 best for wf-5, KM 0.60 best for wf-8), confirming that no single fixed threshold matches both regimes. Adaptive FaaS_{Spot} leads the best fixed variant by 11.0 pp on wf-5 and 5.3 pp on wf-8, validating the Thompson Sampling bandit’s context-dependent selection.

6.9 Summary of Results

Table 6.10 consolidates the key outcomes across all three primary metrics.

Table 6.10: Summary: FaaSpt vs. baselines on all three primary metrics.

Method	Success	Latency	Cost/s	Spot-only?
FaaSpt (FS)	89.0%	126s	\$0.018	Yes (all spot)
MScheduler	81.8%	334s	\$0.040	No (on-demand fallback)
Hourglass	76.2%	506s	\$0.047	No
Snape	58.6%	297s	\$0.050	No (mixed)
Burst-HADS	53.4%	290s	\$0.259	No

FaaSpt leads on all three metrics simultaneously while operating under a stricter constraint (no on-demand fallback) than any baseline. The integrated design — HEFT-based criticality ranking, structural checkpoint classification, priority-ordered checkpoint recovery, and budget-gated Thompson Sampling scaling — achieves 89.03% success at \$0.018 cost per success and 126 s mean latency, outperforming all four baselines on every metric. Checkpoint efficiency (10.6 checkpoints per successful run vs. 39.9 for Hourglass) confirms that placement value dominates frequency. The ablation confirms all three mechanisms are necessary; the sensitivity analysis confirms that adaptive parameter selection outperforms any fixed threshold by 9–17 pp.

6.10 Discussion

6.10.1 Structural Vulnerability and the Checkpoint Placement Effect

The most striking result is the workflow-level variation in FaaSpt’s reliability advantage. On wf-1 and wf-2 (simple topologies), most baselines also achieve near-100% completion and FaaSpt’s margin is negligible. On wf-5 (LIGO, synchronization barrier), FaaSpt reaches 86.2% while all four baselines fall below 41%, a margin of **+43 percentage points**. In wf-5, a single preemption at the Thınca synchronization node blocks both sub-pipeline branches; baselines that lack a checkpoint at this node must restart both branches from scratch. FaaSpt’s Critical-tier checkpoint at Thınca converts a dual-pipeline restart into a single-branch replay.

This structural insight reveals that checkpoint *frequency* is not the operative variable. Hourglass employs 39.9 checkpoints per successful run ($3.8\times$ more than FaaSpt’s 10.6) yet achieves only 76.22% success. Variant A (uniform checkpointing) inflates cost per success from \$0.018 to \$0.033 without commen-

surate reliability gain (86.44% vs 89.03%), because placement specificity—not raw checkpoint density—drives recovery value.

6.10.2 The Cost–Reliability Inversion

A common assumption is that reliability improvements require additional resource consumption. FaaS_{Spot} inverts this: cost per successful run decomposes into the cost of successful runs, failed runs (wasted compute), and recovery overhead. Baselines accumulate high cost-per-success because failed-run compute is wasted on full-workflow restarts. FaaS_{Spot} converts failures into partial sub-DAG replays: \$0.018 vs \$0.040 for MScheduler (2.2×) and \$0.259 for BurstHADS (14.4×). Reliability failure is more expensive than FaaS_{Spot}’s checkpointing and scaling overhead combined.

6.10.3 Latency Concentration and the Full-Restart Cost

Baselines exhibit bimodal latency distributions: runs that escape preemption complete quickly; runs that incur a full-workflow restart produce heavy right tails. FaaS_{Spot}’s distribution is concentrated and unimodal because Critical-tier checkpoints at fan-out boundaries bound recovery to a sub-DAG replay. At p99, FaaS_{Spot} achieves 346 s while Hourglass reaches 2,199 s; cross-workflow variance is smallest for FaaS_{Spot} (± 42 s at p50 vs. ± 143 s for Hourglass), reflecting predictable latency regardless of where preemptions land.

6.10.4 Anomalies and Boundary Cases

wf-4 (Epigenomics): an honest accounting. MScheduler’s success rate exceeds FaaS_{Spot}’s by 2.5 pp (77.1% vs 74.6%), inside the paired binomial half-width of ± 5.3 pp and therefore not statistically distinguishable; FaaS_{Spot} retains a 22% cost-per-success advantage. The most plausible explanation is that Epigenomics consists of several long, largely independent parallel branches converging at a single merge fan-in, and MScheduler’s reactive on-demand migration contains branch-level failures before they propagate to the barrier. We present this explanation as a post-hoc rationalization rather than a structural prediction: the topological signal that would distinguish wf-4 from wf-5 *a priori*—both have merge fan-ins—is not captured by the current criticality score, and the score does not provide a quantitative threshold below which structural checkpoint-and-replay loses to branch-level migration. A structural predictor for this regime (for example, branch independence ratio combined with branch length) is a natural extension and is noted as future work in Chapter [7 §7.4.3](#).

wf-8 (Montage): joint independent-pipeline ceiling. FaaS_{Spot} achieves approximately 50% success on wf-8, comparable to the best baseline. The structural cause is that three independent sub-pipelines must each avoid disruptive preemption for the workflow to succeed: under independent per-VM hazard h and equal sub-pipeline durations, the joint probability is approximately $(1 - h)^3$, which compresses any per-pipeline improvement into a smaller terminal gain. FaaS_{Spot}’s mechanisms still help on the recovery cost (cost-per-success on wf-8 is \$0.051 versus \$0.066 for MScheduler), but the success rate is bounded by infrastructure exposure that no fault-tolerance layer can fully escape without on-demand fallback. This is the regime where the central thesis claim—“pure-spot can match hybrid systems”—reaches its practical limit, and the result is reported as such rather than masked by aggregate statistics. The Limitations chapter §7.1 discusses the relationship between this structural ceiling and the unmodeled capacity-scarcity scenario.

6.10.5 Mechanism Interaction: Why Integration Matters

The ablation confirms that removing any single mechanism degrades *both* reliability and cost efficiency simultaneously. Variant A (uniform checkpointing) achieves 86.44% at \$0.033/success—lower reliability at nearly double the cost. Variant B (no adaptive scaling) degrades on high-preemption workflows as the retry queue grows faster than fixed-replica capacity drains. Variant C (no lineage-based retry) is most damaging at 50.05% and \$0.082/success: checkpoints are written but never consulted. Fault-tolerant spot execution is a coordination problem; solutions addressing any one dimension in isolation yield marginal gains at best.

CHAPTER 7

LIMITATIONS AND FUTURE WORK

Chapter 6 demonstrates that FaaS_{Spot} improves success rate, cost-per-success, and latency under spot-instance volatility. Nevertheless, these findings should be interpreted within the scope conditions of the current system design and evaluation protocol. This chapter formalizes those scope conditions, separates *system limitations* from *threats to validity*, and outlines the future research agenda they motivate.

7.1 System Scope and Core Assumptions

FaaS_{Spot} is intentionally formulated as a pure-spot controller with a closed loop around checkpoint selection, prioritized retry, and adaptive scaling. This design is appropriate for evaluating spot-only reliability engineering; however, it also introduces assumptions that can affect transferability to production environments.

7.1.1 Spot Capacity Availability Is Not Explicitly Modeled

The current control loop assumes that, when the policy issues a scale or recovery action, spot capacity can be reacquired with acceptable delay. In operational cloud markets, spot pools may become unavailable, significantly delayed, or rapidly repriced during regional demand perturbations (Son et al., 2024; Wu et al., 2024). Under such conditions, a policy that is optimal under *expected* capacity access may become suboptimal under realized capacity scarcity.

This limitation is most consequential for long DAGs with high interruption exposure. If replacement instances cannot be provisioned while retry pressure

increases, backlog growth may exceed the controller’s drain capacity. Accordingly, the reported results should be interpreted as performance under interruption volatility *with feasible spot reacquisition*, not as a guarantee under severe market scarcity.

Correlated zone-level preemption is not modeled. A related and equally consequential gap is that the emulator samples each VM’s lifetime independently from the SkyPilot CDF. Real spot environments exhibit *correlated* preemption: entire spot pools (an instance type within an AZ) can drain within minutes when regional demand spikes (Wu et al., 2024). Recent empirical evidence from Kim et al. (Kim et al., 2026) reinforces this: across 68 instance types and 15 regions on AWS and Azure, co-interruptions within the same AZ occur within three minutes in over 92% of cases, confirming that independent lifetime models substantially understate zone-level failure risk. Under correlated drain, the structural advantage of FaaS’s tiered checkpointing is unaffected (the topology has not changed) but the survival-aware scaler can be defeated: scaling out to “safer” instances does no good if every instance in the same pool is reclaimed simultaneously. The reported scaler ablation (Variant B) understates this risk because the emulator does not produce zone-correlated drain events. A scarcity-regime extension is the first item of future work in §7.4.1.

Control-plane preemptibility. As stated in Chapter 4 §4.1, the control plane (MySQL state store, Fission router, scheduler pods, and the three runtime controllers) is assumed to run on non-preemptible capacity. This is the convention in the literature, but it is not the only possible choice. A fully spot-resident control plane would need to handle its own preemption-driven leader election and durable state hand-off, neither of which is in scope for this thesis. The cost impact of the assumption is small (single-digit control-plane pods regardless of workflow scale), but the engineering implication—that “pure-spot” here means “pure-spot task plane”—should not be over-extrapolated.

The Montage (wf-8) result (approximately 50% success across all methods) does not exemplify this scarcity scenario directly. wf-8’s difficulty is structural: three fully independent parallel sub-pipelines must each avoid disruptive preemption, and the joint per-pipeline survival probability multiplies down to roughly $(1 - h)^3$ regardless of how the time axis is scaled (Section 6.10.4). A true scarcity scenario would manifest differently—as rising retry-queue backlog and growing recovery latency even for structurally simple workflows—which is not observed in the evaluation. The scarcity regime nevertheless remains a real production concern that the current controller does not explicitly address.

7.1.2 Survival Modeling Is Built from a Single Availability-Zone Context

The current risk controller relies on spot-lifetime modeling calibrated from a specific availability-zone context. Although this choice improves experimental control, it constrains direct generalization. Spot behavior varies across region, zone, accelerator type, time horizon, and broader market dynamics (Son et al., 2024, Wu et al., 2024, Lee et al., 2022).

Consequently, absolute gains in success rate and cost efficiency may shift under deployment conditions that differ from the calibration context. The qualitative mechanism—risk-aware coordination of checkpointing and scaling—is expected to remain useful, but threshold values and operating points are environment-dependent. In effect, policy *structure* is more transferable than exact numeric *tuning*.

This has a practical implication for deployment: FaaSot’s checkpoint planner and bandit require recalibration when moved to a new region, zone, or instance family. Without recalibration, the survival model may mismatch actual lifetime distributions, and the bandit’s prior may require additional warm-up samples to converge. Section 7.3 discusses a self-calibrating warm-up procedure as a mitigation.

A second concern is *KM-curve drift*: the current implementation fits the survival curve once from the SkyPilot dataset and queries it at runtime, but does not re-fit as new lifetime observations accumulate. In a production deployment where spot dynamics shift over time-of-day or with provider capacity expansions, an unupdated curve would yield biased risk scores. Online refitting (an exponentially-weighted KM estimator, or a sliding-window non-parametric maximum-likelihood update) is the natural mitigation and is left to future work alongside the multi-AZ extension in §7.4.2.

7.2 Threats to Validity

In addition to system-scope assumptions, methodological threats influence how broadly the conclusions can be generalized.

7.2.1 Internal Validity

Internal validity concerns whether observed improvements are causally attributable to the proposed mechanism rather than to hidden confounders. The ablation and sensitivity analyses in Chapter 6 (§6.7–§6.8) reduce this risk by isolating

module contributions and sweeping key parameters. However, residual confounding remains possible because arrival dynamics, interruption clustering, and queue-state coupling can alter controller behavior in ways that are difficult to exhaustively enumerate.

Policy tuning is also necessarily finite. Although key settings were stress-tested, the feasible parameter space exceeds practical experiment budgets. Therefore, the reported gains indicate robust performance within the evaluated regime, not global optimality across all admissible control configurations.

A related concern is that the three sensitivity parameters—budget factor, KM survival threshold, and retry priority weight—are swept independently rather than jointly. The full FaaS_{Spot} bandit co-adapts all three simultaneously; the independent sweeps characterize each parameter’s marginal contribution under partial adaptation. Joint sensitivity analysis across all three parameters is a methodological extension that would provide stronger internal validity evidence; we report it as a known gap rather than ignore it.

Delayed-reward credit assignment. The Adaptive Scaler’s contextual bandit treats the reward $r_t = \Delta\sigma_t/\text{cost}_t$ as the consequence of the single action selected at the start of observation window t . In practice the reward stream is delayed (the success-rate change reflects actions taken across several prior windows) and non-stationary (correlated preemption bursts alter the underlying hazard mid-window). The bandit is therefore a stationary-instantaneous approximation of a non-stationary delayed-reward process. The KM-tier safety floor and the $[-1, 1]$ reward clip limit the damage of any single mis-attributed update, but a worst-case attribution error could shift the reported success rate by up to roughly 1–2 pp on the high-preemption workflows. A principled treatment via off-policy delayed-reward bandit estimation (Duan et al., 2024) is left to future work.

Hyperparameter overfitting risk. The bandit cold-start parameters and KM-tier thresholds were tuned on wf-1 and wf-2 during development and then held fixed for all evaluation workflows including the hardest ones (wf-5, wf-8). This protocol mirrors the standard cross-validation discipline of holding tuning data out from evaluation, but the tuning set is small (2 simple workflows) and shares the structural family of the evaluation suite (scientific DAGs); the residual risk of fortuitous tuning is non-zero. Appendix Table A.7 reports every tuned parameter so that follow-up work can re-tune on a different held-out subset.

7.2.2 Construct Validity

Construct validity concerns whether selected metrics adequately represent system objectives. The evaluation focuses on success rate, total cost, cost-per-success, and latency (including tail behavior shown in the violin plots) because these metrics are operationally salient and aligned with the thesis objective. Nonetheless, they do not fully capture all production-quality attributes.

Important dimensions such as cross-workflow fairness, jitter under burst arrivals, storage durability trade-offs, and long-horizon operational complexity are only partially reflected. Similarly, checkpoint count is informative but remains an indirect proxy for storage-system pressure. Thus, conclusions are strongest for reliability–cost–latency coordination, while broader operational quality remains partially measured.

The cost reporting convention (all experiments priced uniformly at spot rates, including on-demand fallback invocations by baselines) is another construct-validity consideration. This convention is conservative for FaaS_{Spot}: pricing on-demand capacity at spot rates understates what baselines would cost in real deployments, since on-demand is typically $3\text{--}5\times$ more expensive than spot. The reported cost advantage for FaaS_{Spot} is therefore a lower bound on the true economic benefit under realistic billing. However, this choice also means that absolute cost-per-success figures should not be taken as production budget estimates without adjusting for actual instance pricing.

7.2.3 External Validity

External validity addresses generalization beyond the evaluated corpus and operating conditions. The workflow suite covers diverse DAG structures and difficulty profiles, including failure-prone cases (wf-5, wf-8), but no finite benchmark can represent all production graph topologies, all resource mixes, and all cloud-market regimes.

The eight workflows are drawn exclusively from the scientific computing and HPC domains. Generalization to other workload classes—event-driven pipelines, ML training workflows, or streaming analytics—is not directly supported by the current evidence. These workload classes may exhibit different DAG structures, task duration distributions, and checkpoint-utility profiles that alter the effectiveness of FaaS_{Spot}’s structural classifier.

Accordingly, the principal claim should be interpreted as follows: FaaS_{Spot} consistently improves outcomes across the tested workload family and modeled market conditions, and its control architecture appears structurally sound for related settings. It should not be interpreted as a universal guarantee that identi-

cal quantitative margins will hold across all workload classes, regions, or future market epochs without recalibration.

7.3 Practical Implications for Deployment

These limitations do not negate the central contribution; they define the boundary between *validated capability* and *deployment hardening*. For practitioners, the key implication is that FaaS_{Spot} already provides a strong spot-only reliability controller, but production deployment should include explicit safeguards for scarcity, startup delay, and transfer-aware optimization.

A practical rollout strategy is to preserve the current closed-loop strengths while introducing environment-aware controls:

- **Capacity-aware admission and scaling:** integrate risk signals with real-time spot availability and feasibility checks before scale-up commitments (Son et al., 2024, Wu et al., 2024). A scale action that cannot be fulfilled due to capacity scarcity should be demoted or queued rather than silently failing.
- **Startup-latency-aware recovery planning:** include warm-up delay estimates in retry prioritization and deadline-aware policies (Poolal et al., 2016, Monge et al., 2020). Recovery actions dispatched to cold instances should carry a startup delay penalty in the retry priority score.
- **Transfer-aware checkpoint valuation:** extend checkpoint utility scoring to include data-transfer and storage-path costs (Aupy et al., 2016, Kail et al., 2016). The dynamic-programming planner should discount checkpoint value for tasks whose state is large relative to recomputation cost.
- **Policy portability workflow:** recalibrate survival and budget thresholds per region/zone/instance family prior to full rollout (Son et al., 2024, Lee et al., 2022). A brief warm-up phase that collects local lifetime samples before production submission can bootstrap the survival model and reduce bandit warm-up cost.

From an engineering perspective, the current results can be interpreted as a high-confidence operating regime for pure-spot execution under modeled conditions. Under more severe scarcity or heavier data-movement constraints, some degradation in absolute metrics should be expected unless the above safeguards are incorporated.

These limitations motivate a concrete research agenda organized along four axes.

7.4 Future Work

7.4.1 Extreme Failure Regimes and Spot-Unavailable Scenarios

The most immediate extension addresses regimes in which spot capacity is intermittently unavailable at scale. Rather than interpreting repeated reacquisition failures as independent transients, the control loop should infer transition into a scarcity regime and switch to a dedicated policy profile that prioritizes preservation of high-value progress, restricts checkpointing to high-criticality states, and suppresses speculative retries. Detection can be implemented as a sequential probability ratio test on the reacquisition delay distribution.

7.4.2 Multi-AZ, Multi-Region, and Multi-Cloud Spot Scheduling

Spot dynamics are heterogeneous across zones and regions (Son et al., 2024, Wu et al., 2024, Lee et al., 2022); a next-generation controller should extend the KM estimator per availability zone and augment the bandit’s context vector with zone identity. Checkpoint planning becomes a joint placement-and-routing problem: a checkpoint is valuable only if its restore path is timely and cost-efficient under current domain conditions. Multi-cloud orchestration further requires a thin abstraction layer normalizing interruption semantics and billing units across AWS, GCP, and Azure.

7.4.3 Adaptive Workflow Restructuring

The wf-4 (Epigenomics) result motivates adapting workflow graph structure itself when runtime evidence indicates elevated structural risk. Runtime restructuring may include splitting high-risk long tasks, merging low-value fine-grained stages, or reordering non-critical branches to reduce expected recomputation impact. Transformation rules must be provably output-preserving to avoid semantic correctness violations.

7.4.4 Toward Production-Grade Operation

Several engineering-focused extensions improve practical adoption: (1) **auto-calibration** via a small batch of probe workflows that bootstrap the KM estimator and initialize the bandit prior from local lifetime observations; (2) **operator-facing explainability** through structured decision artifacts (risk snapshots,

budget-state traces, counterfactual summaries); (3) **expanded workload coverage** to ML training pipelines, event-driven workflows, and streaming analytics; and (4) **reproducible evaluation packs** publishing the trace-driven emulator, workflow DAG definitions, and baseline implementations as an open benchmark suite.

CHAPTER 8

RELATED WORK

FaaS_{Spot} sits at the intersection of two research areas: fault-tolerant execution on spot (preemptible) cloud instances and serverless DAG workflow execution. Prior work in each area has advanced individual mechanisms, but no existing system addresses the combination that FaaS_{Spot} targets: a pure-spot serverless execution platform that exploits static workflow topology to drive coordinated checkpointing, retry, and scaling without on-demand fallback. This chapter surveys both areas, maps out the design space, and concludes with a comparison table that precisely locates FaaS_{Spot} relative to related systems.

8.1 Survey of Related Work

Checkpoint theory and spot modeling. Young (Young, 1974) and Daly (Daly, 2006) establish optimal checkpoint intervals for sequential jobs; Aupy (Aupy et al., 2016) and Tomar (Tomar et al., 2023) extend placement to DAG workflows and spot environments. FaaS_{Spot} departs from this line by integrating checkpointing with survival-aware scaling and lineage-based retry as a coordinated unit. Spot lifetime characterization work—SpotLake (Lee et al., 2022), SpotCheck (Sharma et al., 2015), SciSpot (Kadupitiya et al., 2022), SkyPilot NSDI’24 (Wu et al., 2024), and the foundational pricing study by Ben-Yehuda et al. (Ben-Yehuda et al., 2013)—shows that lifetimes are non-exponential and heterogeneous across zones, motivating FaaS_{Spot}’s Kaplan-Meier survival estimator (Kaplan and Meier, 1958). Cloud-index tracking (Shastri and Irwin, 2018) further demonstrates that effective spot use requires capturing this heterogeneity in the cost model itself. Recent measurement work by Kim et al. (Kim et al., 2026) introduces a near-zero-cost availability probing method and documents that co-interruptions within the same availability zone occur within three

minutes in over 92% of observed events, empirically corroborating FaaS_{Spot}'s zone-level scarcity concern identified in §7.1.

Hybrid spot/on-demand systems. Hourglass (Joaquim et al., 2019), Snape (F. Yang et al., 2023), MScheduler (Portella et al., 2023), and Burst-HADS (Teylo et al., 2023)—the four evaluation baselines—achieve higher completion rates by maintaining on-demand capacity as a backstop at the cost of on-demand pricing. Tributary (Harlap et al., 2018), SpotOn (Subramanya et al., 2015), and deadline-aware spot schedulers (Tarafdar et al., 2022) follow the same pattern. Related hybrid-scheduling work includes ensemble-workflow scheduling on IaaS mixes (Chen et al., 2024), planning under uncertainty on unreliable cloud resources (Oikonomou et al., 2020), microservice hybrid-resource scheduling with adaptive learning (Gu et al., 2021), cost-deadline constrained robust scheduling (Nag et al., 2024), and real-time volatile-resource workflow scheduling (Wang et al., 2024). Online VM-failure prediction (Saxena and Singh, 2022) provides an alternative reliability signal but requires per-instance training and does not exploit DAG structure. AutoBoT (Varshney and Simmhan, 2019) provides a resilient and cost-effective scheduler for bag-of-tasks workloads on spot VMs using checkpoint-based restart. ExoSphere (Ambati et al., 2020) provides a schedulable parallel-virtualization substrate but still relies on a non-preemptible scheduling layer for reliability. Multi-level checkpointing for distributed deep learning on spot clusters (Cho et al., 2024) reduces checkpoint overhead through a hierarchical approach, though it does not exploit DAG topology to guide checkpoint placement. FaaS_{Spot} achieves superior results through structural mechanisms alone, with no on-demand fallback.

DAG-structure-aware fault tolerance. TR-Spark (Yan et al., 2016) performs lineage-aware checkpointing in Spark's stage model but assumes stable resources for high-criticality stages and cannot be applied to fine-grained serverless execution. Pado (Y. Yang et al., 2017) exploits topology to migrate critical tasks to reserved nodes; FaaS_{Spot} replaces migration with checkpoint-and-retry since no reserved nodes exist. SpotVerse (Son et al., 2024) performs cross-region placement for bioinformatics workflows but does not direct task-level checkpoint or retry decisions, and SkyNomad (Li et al., 2026) similarly exploits multi-region spot capacity at the job level rather than at internal DAG boundaries.

Transient resources for analytics, ML serving, and training. Flint (Sharma et al., 2016), Portfolio (Sharma et al., 2017), Bamboo (Thorpe et al., 2023), and Parcae (Duan et al., 2024) target transient resources for analytics or ML training, while SpotTune (Servia-Rodríguez et al., 2020) explores cost-efficient hyper-parameter tuning on the same hardware class. SpotServe (Miao et al., 2024) and SkyServe (Mao et al., 2025) extend the model to online LLM

serving on preemptible instances, and GFS (Duan et al., 2025) introduces preemption-aware scheduling for GPU spot clusters. SpotWeb (Sharma et al., 2019) and the mixed-workload optimization of Singh et al. (Singh et al., 2017) extend transient-resource techniques to latency-sensitive and mixed workloads. Complementary cost-aware training work such as FedCostAware (Sinha et al., 2025) applies cloud spot-market awareness to federated learning by scheduling client training rounds against instance-price dynamics; its cost-aware objective is aligned with FaaSpt’s cost-per-success formulation, though it targets a different workload class (federated aggregation) and does not model DAG-level cascade failures. Bamboo and Parcae deserve a sharper positioning because they share two ideas with FaaSpt: lineage-based recovery and predictive provisioning on preemptible capacity. Bamboo introduces *redundant computation* across pipeline stages to recover from ML-training preemptions without checkpointing; Parcae extends this with a predictive component that anticipates preemption and reconfigures the pipeline ahead of time. Both treat the workload as a pipeline of homogeneous training iterations, and the recovery unit is the iteration boundary. FaaSpt differs in three concrete respects. *First*, the recovery unit is a structurally classified DAG position, not a homogeneous iteration boundary; the tier (CRITICAL/MEDIUM/LOW) is derived from topology, not from training-loop semantics. *Second*, the predictive signal is statistical (Kaplan–Meier survival) rather than utilization- or loss-curve-based, which is the only signal available for general scientific DAGs whose tasks do not have a single dominant convergence metric. *Third*, FaaSpt’s coordination is offline-then-online: a single classification pass drives three online mechanisms, whereas Bamboo and Parcae continuously re-plan the redundant-compute schedule. The overlap with these systems is real and should not be understated; the contribution of FaaSpt is to bring this class of ideas to topology-aware DAG fault tolerance.

Serverless workflows and platforms. Wukong (Carver et al., 2020) and its companion DAG-engine study (Carver et al., 2019), Speedo (Daw et al., 2021), Mashup (Roy, Patel, Gadepally, and Tiwari, 2022), DayDream (Roy, Patel, and Tiwari, 2022), StarShip (Basu Roy and Tiwari, 2024), HyperFlow (Malawski et al., 2020), Pronghorn (Kohli et al., 2024), Nightcore (Jia and Witchel, 2021), Trident (Zhu et al., 2025), and Bilal et al. (Bilal et al., 2023) optimize dispatch, locality, or resource sizing on reliable infrastructure without addressing spot preemption. ORION (Mahgoub, Barsallo Yi, Shankar, Elnikety, et al., 2022) and WiseFuse (Mahgoub, Barsallo Yi, Shankar, Minocha, et al., 2022) optimize serverless DAG execution through DAG-aware resource sizing and workload transformation using production traces. Netherite (Burckhardt et al., 2022) demonstrates that group-commit state persistence achieves order-

of-magnitude throughput improvements, motivating FaaS_{Spot}’s choice to concentrate checkpoint I/O at structurally critical positions rather than every task. Jolteon (Zhang et al., 2024) configures resource provisioning via a stochastic performance model that bounds tail latency under function variability; Metis (Tang et al., 2025) introduces a non-clairvoyant, workflow-aware OS scheduler that treats serverless workflows as first-class scheduling entities (SoCC 2025). GridGreen (Samanta et al., 2025) integrates serverless computing into HPC environments for hybrid execution and sustainability. SeBS-Flow (Schmid et al., 2025) provides the first platform-agnostic serverless workflow benchmarking suite, enabling consistent evaluation across platforms. Cost-of-serverless analyses (Liu and Niu, 2024) and scheduling algorithms for serverless (Majewski et al., 2021) further inform FaaS_{Spot}’s cost model. Speedo specifically targets serverless DAG dispatch ordering and Nightcore targets sub-millisecond function chains; both are orthogonal to and composable with FaaS_{Spot}’s fault-tolerance layer, since they assume a non-preemptible substrate and FaaS_{Spot} provides one (through tiered checkpoint-and-replay) on top of preemptible hardware.

Workflow management frameworks. Pegasus (Deelman et al., 2015, 2019) and Ray (Moritz et al., 2018) represent the broader workflow-management context in which FaaS_{Spot}’s contributions sit. The Adaptive Scaler’s bandit controller is grounded in Thompson Sampling theory (Agrawal and Goyal, 2013), extended here with a Kaplan–Meier survival safety floor that constrains the action space before the bandit optimizes within it.

8.2 Comparison and Positioning

Table 2.1 (Chapter 2) maps all surveyed systems along six design dimensions. Three observations follow. First, no prior system simultaneously occupies the spot-only + DAG-aware cell: TR-Spark and Pado exploit DAG structure but depend on non-spot resources for protection; transient-resource systems (Flint, Bamboo, Portfolio) operate on spot or transient resources but do not exploit workflow topology. FaaS_{Spot} occupies this previously empty cell.

Second, partial sub-DAG replay (PR recovery) requires structural awareness: to resume from a fan-out boundary checkpoint rather than from the workflow origin, the system must know where fan-out boundaries are. Systems with full-workflow restart (FW) cannot improve recovery latency by structural checkpointing. The ablation study (Chapter 6, §6.7) quantifies this cost directly: removing lineage-based replay drops aggregate success from 89.03% to 50.05%.

Third, survival-aware scaling—proactively provisioning capacity before a preemption occurs—is unique to FaaS_{Spot} in this comparison. Most systems scale reactively, responding to a preemption event after it has already disrupted progress. Proactive scaling under survival risk reduces recovery queueing delay, which the latency analysis in Chapter 6 shows is a primary contributor to the latency advantage on complex workflows.

CHAPTER 9

CONCLUSION

Cloud spot instances provide substantial cost advantages, but interruption volatility makes reliable workflow execution challenging for dependency-rich DAGs. This thesis addressed whether high reliability and cost efficiency can be sustained on *pure-spot* infrastructure without on-demand fallback. The central answer is yes—but only when offline structure-aware planning is tightly coupled with runtime adaptive control.

The headline measurements are reported throughout Chapter 6: FaaS_{Spot} leads on success rate, cost per success, and mean and tail latency across the eight-workflow suite, with the widest margin on structurally complex workflows (a +43 pp success-rate advantage on wf-5 LIGO). The ablation localizes the reliability contribution to lineage-based retry, the cost contribution to selective structural checkpointing, and the high-preemption-regime contribution to survival-aware scaling; none of the three mechanisms is sufficient on its own. Two boundary cases (wf-4 Epigenomics, where reactive on-demand migration is competitive on success rate; wf-8 Montage, where the joint independent-sub-pipeline survival probability caps any single-pipeline gain) mark the practical limits of the current design and are reported as such rather than smoothed over by aggregate statistics. The full evaluation, including a real-cloud validation pass under uncompressed wall-clock time, is identified in Chapter 7 as the principal external-validity threat and the most urgent follow-up work.

9.1 Final Perspective

The central takeaway is that cost-efficient reliability on volatile cloud resources does not emerge from checkpointing alone, scaling alone, or prediction alone. It emerges when planning, recovery, and resource control are intentionally coupled around workflow structure and runtime risk. Future cloud workflow sys-

tems should be designed as integrated control systems—explicitly balancing survivability, cost, and latency under uncertainty—rather than as collections of loosely connected single-objective mechanisms.

APPENDIX A

SUPPLEMENTARY EVALUATION AND REPRODUCIBILITY DETAILS

This appendix provides compact supporting material for the main results in Chapter 5 (Implementation) and Chapter 6 (Evaluation). The goal is to make experimental choices, measurement definitions, and headline outcomes easy to audit without interrupting the flow of the primary narrative.

All figures in this appendix are derived from the same eight-workflow campaign used throughout the main text: 500 invocations per workflow under matched trace-replay conditions, $60\times$ time compression, SkyPilot Wu et al., 2024 p3. 2xlarge lifetime CDFs sampled in $[p_{10}, p_{90}]$ for preemption modeling, and SpotLake Lee et al., 2022 m3. 2xlarge spot prices applied uniformly to all methods for cost accounting (the two-instance-type convention is justified in Chapter 5 §5.4). Numbers in this appendix are consistent with Chapter 6; any discrepancy with earlier draft snapshots is a typographical error and should be reported.

A.1 Reproducibility Checklist

All experiments follow a trace-driven protocol designed for controlled comparison across methods. The following controls are held constant unless explicitly varied in a sensitivity analysis:

- identical workflow inputs and submission profiles across methods (Poisson arrivals at $\lambda = 1/s$, $N = 500$ submissions per workflow),

- identical spot interruption and price trace sources for each paired trial (one CDF per AWS instance type and AZ, with per-pod lifetimes drawn from $[p_{10}, p_{90}]$ at pod creation),
- identical emulator tick policy and cost-accounting logic ($\Delta_{\text{emu}} = 60\Delta_{\text{real}}$, $h_{\text{emu}} = \Delta_{\text{real}}/60$ hours per billing tick),
- identical runtime environment (Kubernetes, Fission, MySQL polling at 5 s, 2 s for the two longest workflows),
- identical baseline evaluation horizon and completion criteria (terminal state: COMPLETE or ABANDONED after exhausting per-UID retry budget).

This ensures that measured differences primarily reflect controller behavior rather than differences in workload sampling or infrastructure conditions. The known limitations of this protocol (single AZ / instance type, lifetime-tail truncation, real-time control plane vs. compressed emulated time) are discussed in Chapter 7 §7.1 and §7.2.

A.2 Workflow and Baseline Inventory

Table A.1 summarizes the eight-workflow evaluation suite used in the main text. It spans narrow vs. wide DAGs, short vs. long critical paths, and varied fan-in/fan-out patterns so that resilience claims are not tied to one graph family.

Table A.1: Workflow inventory used across the evaluation suite (consistent with Chapter 6 Table 6.2).

Workflow	Type	Task count	Role in evaluation
wf-1	Simple Pipeline (synthetic)	5	Low-parallelism sequential baseline; sanity check.
wf-2	Fork-Join (synthetic)	7	Single-barrier aggregation behavior.
wf-3	1000 Genome	11	Cross-branch data dependency.
wf-4	Epigenomics	31	Long parallel-pipeline merge fan-in.
wf-5	LIGO	36	Dual sub-pipeline with Thincas synchronization barrier.
wf-6	SRASearch	15	Dual fan-out with nested fan-in.
wf-7	CyberShake	42	Multi-root cascade propagation to single sink.
wf-8	Montage	46	Triple parallel sub-pipelines with sequential post-processing.

Table A.2 lists the compared methods and the role each plays in the comparison.

Table A.2: Baseline inventory and comparison role.

Method	Category	Comparison purpose
FaaSpt (FS)	Proposed	Integrated pure-spot closed-loop controller.
MScheduler Portella et al., 2023	SOTA	Spot-first with reactive on-demand migration.
Hourglass Joaquim et al., 2019	SOTA	Deadline-pacing transient-resource scheduler.
Snape F. Yang et al., 2023	SOTA	Mixed-instance reliability-target scheduler.
Burst-HADS Teylo et al., 2023	SOTA	Bag-of-task spot-burst scheduler (workload-mismatch reference).
FaaSpt-A (no selective ckpt)	Ablation	Uniform CRITICAL-tier checkpointing.
FaaSpt-B (no scaler)	Ablation	Fixed replica counts from offline plan.
FaaSpt-C (no lineage retry)	Ablation	Full-workflow restart on every failure.

A.3 Key Aggregate Results Snapshot

For quick reference, Table A.3 consolidates the main aggregate outcomes reported in Chapter 6.

Table A.3: Aggregate performance snapshot across all eight workflows (mirrors Tables 6.3, 6.4, and 6.5).

Method	Success rate (%)	Cost/success (\$)	Avg. latency (s)
FaaSpt (FS)	89.03	0.018	126
MScheduler	81.78	0.040	334
Hourglass	76.22	0.047	506
Snape	58.56	0.050	297
Burst-HADS	53.43	0.259	290
FaaSpt-A (uni-form ckpt)	86.44	0.033	—
FaaSpt-B (no scaler)	86.74	0.041	—
FaaSpt-C (no lineage)	50.05	0.082	—

Table A.4 summarizes the interruption-heavy subset (wf-5 LIGO, wf-7 CyberShake, wf-8 Montage), where resilience quality is most visibly differentiated.

Table A.4: Hard-workflow summary on the three workflows with the widest cascade exposure.

Quantity	Value
FaaSpt success rate (hard subset)	78.6%
FaaSpt cost per successful run (hard subset)	\$0.040
FaaSpt average latency (hard subset)	314 s
Success gain vs. best SOTA on wf-5	+43.0 pp (86.2% vs. 41%)
Latency reduction vs. best SOTA on wf-5	4.2×
Cost/success reduction vs. best SOTA on wf-5	~2.0×

A.4 Per-Workflow Result Tables

Table A.5 reports per-workflow success rate (%) for the proposed system and the four SOTA baselines. Per-workflow standard errors are reported with the result rather than aggregated, so that wf-4 (where the proposed system trails MScheduler within standard error) is visibly identifiable.

Table A.5: Per-workflow success rate (%). Best per row in bold; ties within ± 2.6 pp (the binomial-proportion 95% half-width at $\hat{p} = 0.90$, $n = 500$) marked with †.

WF	FaaS _{Spot}	MS	HG	SN	BH
wf-1	99.8	99.6†	95.4	88.0	90.6
wf-2	100.0 †	99.6†	88.0	71.0	84.8
wf-3	100.0	93.7	89.4	71.4	54.6
wf-4	74.6	77.1 †	71.2	41.8	27.8
wf-5	86.2	41.0	35.4	28.8	16.6
wf-6	100.0	83.4	61.6	44.4	36.0
wf-7	99.7	98.2	87.6	74.9	57.2
wf-8	49.7	61.7	75.6	48.2	60.0

Table A.6 reports per-workflow cost per successful run.

Table A.6: Per-workflow cost per successful run (USD; lower is better). Best per row in bold.

WF	FaaS _{Spot}	MS	HG	SN	BH
wf-1	0.004	0.026	0.022	0.029	0.084
wf-2	0.006	0.018	0.029	0.025	0.092
wf-3	0.009	0.024	0.031	0.033	0.140
wf-4	0.055	0.067	0.078	0.084	0.530
wf-5	0.022	0.067	0.082	0.071	0.310
wf-6	0.010	0.046	0.056	0.061	0.290
wf-7	0.011	0.036	0.045	0.041	0.205
wf-8	0.051	0.066	0.034	0.058	0.420

A.5 Metric Definitions Used in Analysis

The core metrics are defined as follows:

- **Workflow success rate**

$$\text{SuccessRate} = \frac{N_{\text{completed}}}{N_{\text{submitted}}},$$

where a submission is `COMPLETED` if and only if the workflow reaches its sink task within the per-UID retry budget.

- **Execution cost**

$$\text{Cost} = \sum_{k \in \mathcal{R}} p_k \cdot t_k,$$

where p_k is the SpotLake unit price for resource k at the billing tick and t_k is active time (in h_{emu} units). The sum runs over all pods active during the experiment, including pods used for failed runs.

- **Cost per successful run**

$$\text{CostPerSuccess} = \frac{\text{TotalCost}}{N_{\text{completed}}}.$$

- **Latency** End-to-end wall-clock real-time completion time per workflow run, measured from invocation to terminal completion at the Fission router. Reported as mean, p_{50} , p_{95} , and p_{99} over successful runs. *Latency is in real-time wall-clock seconds, not emulated time*, because the Adaptive Scaler, Retry Manager, and detection polling all run in real time.
- **Checkpoints per success**

$$\text{Ckpts/Success} = \frac{N_{\text{checkpoints written}}}{N_{\text{completed}}},$$

a measure of how productively the I/O budget is spent.

A.6 Bandit and Survival Hyperparameters

All bandit and KM-tier hyperparameters used in evaluation are summarized below. Cold-start values were chosen on the two simplest workflows (wf-1 and wf-2) and held fixed for the remaining workflows; the budget thresholds and KM cut points are derived from inspection of the SkyPilot CDF and are reported here for full reproducibility (see Chapter 4 §4.4 for the methodological justification).

Table A.7: Bandit and KM hyperparameters used in all evaluations.

Parameter	Value	Role
KM <code>SAFE</code> threshold	0.55	Suppresses bandit
KM <code>PROACTIVE</code> threshold	0.50	Light scale-out
KM <code>WARNING</code> threshold	0.40	Moderate scale-out
BLR regularization λ_{reg}	1.0	Prior precision matrix
Cold-start ε	0.5	Exploration on {idle, conservative}
Cold-start ticks T_{cold}	10	Bandit cold-start length (complex DAGs)
Simple-DAG cold start	30 real min	Effectively passive under $60\times$ compression
Aggressive-action cooldown	120 s	Prevents high-cost decision repetition
Reward clip	$[-1, 1]$	Outlier mitigation
f_{cap} (per-task scaling cap)	$2.5\times$	Caps offline amplification
Pods per emulated machine	3	Models correlated VM-level preemption
Trace lifetime sampling	$\text{Unif}(p_{10}, p_{90})$	Tail-truncated by design

A.7 Notes for Future Extensions

This appendix focuses on compact reproducibility and result snapshots. Future versions of the evaluation pack can include expanded per-workflow raw tables, complete sensitivity-sweep matrices, additional failure-trace diagnostics, and a live-cloud validation pass on AWS `p3.2xlarge` spot instances under real (uncompressed) wall-clock time — the validation pass that Chapter 7 identifies as the principal external-validity threat to the current results.

BIBLIOGRAPHY

- [1] Kaplan, E. L., & Meier, P. (1958). Nonparametric estimation from incomplete observations. *Journal of the American Statistical Association*, 53(282), 457–481. <https://doi.org/10.1080/01621459.1958.10501452>
- [2] Young, J. W. (1974). A first order approximation to the optimum checkpoint interval. *Communications of the ACM*, 17(9), 530–531. <https://doi.org/10.1145/361147.361115>
- [3] Auer, P., Cesa-Bianchi, N., & Fischer, P. (2002). Finite-time analysis of the multiarmed bandit problem. *Machine Learning*, 47(2), 235–256. <https://doi.org/10.1023/A:1013689704352>
- [4] Topcuoglu, H., Hariri, S., & Wu, M.-Y. (2002). Performance-effective and low-complexity task scheduling for heterogeneous computing. *IEEE Transactions on Parallel and Distributed Systems*, 13(3), 260–274. <https://doi.org/10.1109/71.993206>
- [5] Daly, J. T. (2006). A higher order estimate of the optimum checkpoint interval for restart dumps. *Future Generation Computer Systems*, 22(3), 303–312. <https://doi.org/10.1016/j.future.2004.11.016>
- [6] Agrawal, S., & Goyal, N. (2013). Thompson Sampling for Contextual Bandits with Linear Payoffs. *International Conference on Machine Learning (ICML)*.
- [7] Ben-Yehuda, O. A., Ben-Yehuda, M., Schuster, A., & Tsafrir, D. (2013). Deconstructing Amazon EC2 spot instance pricing. *ACM Transactions on Economics and Computation*, 1(3), 1–20. <https://doi.org/10.1145/2509413.2509416>
- [8] Zheng-Bradley, X., & Flicek, P. (2013). The 1000 genomes project: Paradigm for a data management plan in a large genomic study. *2013 International Conference on Parallel and Distributed Systems (ICPADS)*, 636–641. <https://doi.org/10.1109/ICPADS.2013.114>
- [9] Pandey, S., Sammut, L., Calheiros, R. N., Melatos, A., & Buyya, R. (2014). Scalable deployment of a LIGO physics application on public clouds: Workflow engine and resource provisioning techniques. In *Cloud com-*

- puting for data-intensive applications* (pp. 3–25). Springer. https://doi.org/10.1007/978-1-4939-1905-5_1
- [10] Poola, D., Ramamohanarao, K., & Buyya, R. (2014). Fault-tolerant workflow scheduling using spot instances on clouds. *Procedia Computer Science*, 29, 523–533. <https://doi.org/10.1016/j.procs.2014.05.047>
 - [11] Auton, A., Abecasis, G. R., Altshuler, D. M., Durbin, R. M., Bentley, D. R., Chakravarti, A., Clark, A. G., Donnelly, P., Eichler, E. E., Flicek, P., et al. (2015). A global reference for human genetic variation. *Nature*, 526(7571), 68–74. <https://doi.org/10.1038/nature15393>
 - [12] Deelman, E., Vahi, K., Juve, G., Rynge, M., Callaghan, S., Maechling, P. J., Mayani, R., Chen, W., Ferreira da Silva, R., Livny, M., & Wenger, K. (2015). Pegasus, a workflow management system for science automation. *Future Generation Computer Systems*, 46, 17–35. <https://doi.org/10.1016/j.future.2014.10.008>
 - [13] Sharma, P., Lee, S., Guo, T., Irwin, D., & Shenoy, P. (2015). SpotCheck: Designing a derivative IaaS cloud on the spot market. *Proceedings of the Tenth European Conference on Computer Systems*. <https://doi.org/10.1145/2741948.2741953>
 - [14] Subramanya, S., Guo, T., Sharma, P., Irwin, D., & Shenoy, P. (2015). SpotOn: A batch computing service for the spot market. *Proceedings of the Sixth ACM Symposium on Cloud Computing*, 329–341. <https://doi.org/10.1145/2806777.2806851>
 - [15] Aupy, G., Benoit, A., Casanova, H., & Robert, Y. (2016). Checkpointing strategies for scheduling computational workflows. *International Journal of Networking and Computing*, 6(1), 2–26. https://doi.org/10.15803/ijn.6.1_2
 - [16] Kail, E., Kacsuk, P., & Kozlovsky, M. (2016). A novel adaptive checkpointing method based on information obtained from workflow structure. *Computer Science*, 17(3), 387. <https://doi.org/10.7494/csci.2016.17.3387>
 - [17] Poola, D., Ramamohanarao, K., & Buyya, R. (2016). Enhancing reliability of workflow execution using task replication and spot instances. *ACM Trans. Auton. Adapt. Syst.*, 10(4). <https://doi.org/10.1145/2815624>
 - [18] Sharma, P., Guo, T., He, X., Irwin, D., & Shenoy, P. (2016). Flint: Batch-interactive data-intensive processing on transient servers. *Proceedings of the Eleventh European Conference on Computer Systems*. <https://doi.org/10.1145/2901318.2901319>

- [19] Yan, Y., Gao, Y., Chen, Y., Guo, Z., Chen, B., & Moscibroda, T. (2016). TR-Spark: Transient computing for big data analytics. *Proceedings of the Seventh ACM Symposium on Cloud Computing*, 484–496. <https://doi.org/10.1145/2987550.2987576>
- [20] Amazon Web Services. (2017, November). *Amazon EC2 spot introduces new pricing model and the ability to launch spot instances via RunInstances API*. Retrieved March 18, 2026, from <https://aws.amazon.com/about-aws/whats-new/2017/11/amazon-ec2-spot-introduces-new-pricing-model-and-the-ability-to-launch-new-spot-instances-via-runinstances-api/>
- [21] Sharma, P., Irwin, D., & Shenoy, P. (2017). Portfolio-driven resource management for transient cloud servers. *Proc. ACM Meas. Anal. Comput. Syst.*, 1(1). <https://doi.org/10.1145/3084442>
- [22] Singh, R., Sharma, P., Irwin, D., Shenoy, P., & Ramakrishnan, K. K. (2017). Optimizing the cost of executing mixed interactive and batch workloads on transient VMs. *Proceedings of the ACM on Measurement and Analysis of Computing Systems*, 1(2), 1–29. <https://doi.org/10.1145/3154499>
- [23] Yang, Y., Kim, G.-W., Song, W. W., Lee, Y., Chung, A., Qian, Z., Cho, B., & Chun, B.-G. (2017). Pado: A data processing engine for harnessing transient resources in datacenters. *Proceedings of the Twelfth EuroSys Conference (EuroSys '17)*, 575–588. <https://doi.org/10.1145/3064176.3064181>
- [24] Cai, Z., Li, X., Ruiz, R., & Li, Q. (2018). Price forecasting for spot instances in cloud computing. *Future Generation Computer Systems*, 79, 38–53.
- [25] Harlap, A., Chung, A., Tumanov, A., Ganger, G. R., & Gibbons, P. B. (2018). Tributary: Spot-dancing for elastic services with latency SLOs. *2018 USENIX Annual Technical Conference (USENIX ATC '18)*. <https://www.usenix.org/conference/atc18/presentation/harlap>
- [26] Moritz, P., Nishihara, R., Wang, S., Tumanov, A., Liaw, R., Liang, E., Elibol, M., Yang, Z., Paul, W., Jordan, M. I., & Stoica, I. (2018). Ray: A distributed framework for emerging AI applications. *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI 18)*, 561–577. <https://www.usenix.org/conference/osdi18/presentation/moritz>
- [27] Shastri, S., & Irwin, D. (2018). Cloud index tracking: Enabling predictable costs in cloud spot markets. *Proceedings of the ACM Symposium on Cloud Computing*, 451–463.

- [28] Carver, B., Zhang, J., Wang, A., & Cheng, Y. (2019). In search of a fast and efficient serverless DAG engine. *2019 IEEE/ACM Fourth International Parallel Data Systems Workshop (PDSW)*, 1–10. <https://doi.org/10.1109/PDSW49588.2019.00005>
- [29] Deelman, E., Vahi, K., Rynge, M., Mayani, R., da Silva, R. F., Papadimitriou, G., & Livny, M. (2019). The evolution of the Pegasus workflow management software. *Computing in Science and Engineering*, 21(4). <https://doi.org/10.1109/MCSE.2019.2919690>
- [30] Joaquim, P., Bravo, M., Rodrigues, L., & Matos, M. (2019). Hourglass: Leveraging transient resources for time-constrained graph processing in the cloud. *Proceedings of the Fourteenth EuroSys Conference 2019 (EuroSys '19)*. <https://doi.org/10.1145/3302424.3303964>
- [31] Jonas, E., Schleier-Smith, J., Sreekanti, V., Tsai, C.-C., Khandelwal, A., Pu, Q., Shankar, V., Carreira, J., Krauth, K., Yadwadkar, N. J., Gonzalez, J. E., Popa, R. A., Stoica, I., & Patterson, D. A. (2019). *Cloud programming simplified: A Berkeley view on serverless computing* (tech. rep. No. UCB/EECS-2019-3). EECS Department, University of California, Berkeley.
- [32] Martinez, R. G., Lopes, A., & Rodrigues, L. (2019). Planning workflow executions when using spot instances in the cloud. *Proceedings of the 34th ACM/SIGAPP Symposium on Applied Computing*, 310–317. <https://doi.org/10.1145/3297280.3297313>
- [33] Sharma, P., Ali-Eldin, A., & Shenoy, P. (2019). SpotWeb: Running latency-sensitive distributed web services on transient cloud servers. *Proceedings of the 28th International Symposium on High-Performance Parallel and Distributed Computing (HPDC '19)*, 1–12. <https://doi.org/10.1145/3307681.3325402>
- [34] Vahi, K., Mayani, R., Mandal, A., Ferreira da Silva, R., & Deelman, E. (2019). Distributed workflow management for LIGO data analysis at scale. *2019 15th International Conference on eScience (eScience)*, 179–188. <https://doi.org/10.1109/eScience.2019.00027>
- [35] Varshney, P., & Simmhan, Y. (2019). AutoBoT: Resilient and cost-effective scheduling of a bag of tasks on spot VMs. *IEEE Transactions on Parallel and Distributed Systems*, 30(7), 1512–1527. <https://doi.org/10.1109/TPDS.2018.2889851>
- [36] Ambati, P., Goiri, Í., Frujeri, F., Gun, A., Wang, K., Dolan, B., Corell, B., Pasupuleti, S., Moscibroda, T., Elnikety, S., Fontoura, M., & Bianchini, R. (2020). ExoSphere: A schedulable parallel virtualization substrate for spot datacenters. *14th USENIX Symposium on Operating Systems Design*

- and Implementation (OSDI '20), 755–770. <https://www.usenix.org/conference/osdi20/presentation/ambati>
- [37] Carver, B., Zhang, J., Wang, A., Anwar, A., Wu, P., & Cheng, Y. (2020). Wukong: A scalable and locality-enhanced framework for serverless parallel computing. *Proceedings of the 11th ACM Symposium on Cloud Computing (SoCC '20)*, 1–15. <https://doi.org/10.1145/3419111.3421286>
 - [38] Khandelwal, V., Chaturvedi, A. K., & Gupta, C. P. (2020). Amazon EC2 spot price prediction using regression random forests. *IEEE Transactions on Cloud Computing*, 8(1), 59–72. <https://doi.org/10.1109/TCC.2017.2780159>
 - [39] Malawski, M., Gajek, A., Zima, A., Balis, B., & Figiela, K. (2020). Serverless execution of scientific workflows: Experiments with HyperFlow, AWS Lambda and Google Cloud Functions. *Future Generation Computer Systems*, 110, 502–514. <https://doi.org/10.1016/j.future.2017.10.029>
 - [40] Monge, D. A., Pacini, E., Mateos, C., Alba, E., & García Garino, C. (2020). CMI: An online multi-objective genetic autoscaler for scientific and engineering workflows in cloud infrastructures with unreliable virtual machines. *Journal of Network and Computer Applications*, 149, 102464. <https://doi.org/10.1016/j.jnca.2019.102464>
 - [41] Oikonomou, P., Kolomvatsos, K., Tziritas, N., Theodoropoulos, G., Loukopoulos, T., & Stamoulis, G. (2020). Uncertainty driven workflow scheduling using unreliable cloud resources. *2020 IEEE 19th International Symposium on Network Computing and Applications (NCA)*, 1–8. <https://doi.org/10.1109/NCA51143.2020.9306707>
 - [42] Servia-Rodríguez, S., Lee, Y. S., Mao, S., Madhyastha, H. V., Lo, J. C., & Madhyastha, Y. C. (2020). SpotTune: Leveraging transient resources for cost-efficient hyper-parameter tuning in the public cloud. *2020 IEEE 40th International Conference on Distributed Computing Systems (ICDCS)*, 1003–1013. <https://doi.org/10.1109/ICDCS47774.2020.00009>
 - [43] Daw, N., Bellur, U., & Kulkarni, P. (2021). Speedo: Fast dispatch and orchestration of serverless workflows. *Proceedings of the ACM Symposium on Cloud Computing (SoCC '21)*, 585–599. <https://doi.org/10.1145/3472883.3486982>
 - [44] Gu, H., Li, X., Liu, M., & Wang, S. (2021). Scheduling method with adaptive learning for microservice workflows with hybrid resource provisioning. *International Journal of Machine Learning and Cybernetics*, 12(10), 3037–3048. <https://doi.org/10.1007/s13042-021-01396-4>
 - [45] Jia, Z., & Witchel, E. (2021). Nightcore: Efficient and scalable serverless computing for latency-sensitive, interactive microservices. *Proceedings*

- of the 26th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '21), 152–166. <https://doi.org/10.1145/3445814.3446701>
- [46] Majewski, M., Pawlik, M., & Malawski, M. (2021). Algorithms for scheduling scientific workflows on serverless architecture. *2021 IEEE/ACM 21st International Symposium on Cluster, Cloud and Internet Computing (CC-Grid)*, 782–789. <https://doi.org/10.1109/CCGrid51090.2021.00095>
 - [47] Burckhardt, S., Chandramouli, B., Gillum, C., Justo, D., Kallas, K., McMahon, C., Meiklejohn, C., & Zhu, X. (2022). Netherite: Efficient execution of serverless workflows. *Proceedings of the VLDB Endowment*, 15(8), 1591–1604. <https://doi.org/10.14778/3529337.3529344>
 - [48] Ferreira da Silva, R., Filgueira, R., Pietri, I., Jiang, M., Sakellariou, R., & Deelman, E. (2022). Pegasus and Epigenomics: Workflow management for large-scale genomics analysis. *Bioinformatics*, 38(3), 866–872. <https://doi.org/10.1093/bioinformatics/btab784>
 - [49] Kadupitiya, J., Jadhao, V., & Sharma, P. (2022). SciSpot: Scientific computing on temporally constrained cloud preemptible VMs. *IEEE Transactions on Parallel and Distributed Systems*, 33(12), 3575–3588. <https://doi.org/10.1109/TPDS.2022.3157272>
 - [50] Lee, S., Hwang, J., & Lee, K. (2022). Spotlake: Diverse spot instance dataset archive service, 242–255. <https://doi.org/10.1109/IISWC55918.2022.00029>
 - [51] Mahgoub, A., Barsallo Yi, E., Shankar, K., Elnikety, S., Chaterji, S., & Bagchi, S. (2022). ORION and the three rights: Sizing, bundling, and prewarming for serverless DAGs. *16th USENIX Symposium on Operating Systems Design and Implementation (OSDI '22)*, 303–320. <https://www.usenix.org/conference/osdi22/presentation/mahgoub>
 - [52] Mahgoub, A., Barsallo Yi, E., Shankar, K., Minocha, E., Elnikety, S., Bagchi, S., & Chaterji, S. (2022). WISEFUSE: Workload characterization and DAG transformation for serverless workflows. *ACM SIGMETRICS Conference (SIGMETRICS)*.
 - [53] Roy, R. B., Patel, T., Gadepally, V., & Tiwari, D. (2022). Mashup: Making serverless computing useful for HPC workflows via hybrid execution. *Proceedings of the 27th ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming (PPoPP '22)*, 46–60. <https://doi.org/10.1145/3503221.3508407>
 - [54] Roy, R. B., Patel, T., & Tiwari, D. (2022). DayDream: Executing dynamic scientific workflows on serverless platforms with hot starts. *SC22*:

- International Conference for High Performance Computing, Networking, Storage and Analysis*, 1–18. <https://doi.org/10.1109/SC41404.2022.00027>
- [55] Saxena, D., & Singh, A. K. (2022). OFP-TM: An online VM failure prediction and tolerance model towards high availability of cloud computing environments. *The Journal of Supercomputing*, 78, 8003–8024. <https://doi.org/10.1007/s11227-021-04235-z>
- [56] Tarafdar, A., Das, R. K., & Khatua, S. (2022). Deadline-sensitive workflow scheduling with on-demand and spot instances in the cloud. *2022 OITS International Conference on Information Technology (OCIT)*, 474–479. <https://doi.org/10.1109/OCIT56763.2022.00094>
- [57] Bilal, M., Canini, M., Fonseca, R., & Rodrigues, R. (2023). With great freedom comes great opportunity: Rethinking resource allocation for serverless functions. *Proceedings of the Eighteenth European Conference on Computer Systems (EuroSys '23)*, 381–397. <https://doi.org/10.1145/3552326.3567506>
- [58] Portella, F. A., Estrela, P. J. B., Malini, R. Q., Teylo, L., Berral, J. L., & Drummond, L. M. A. (2023). MScheduler: Leveraging spot instances for high-performance reservoir simulation in the cloud. *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*. <https://doi.org/10.1109/CloudCom59040.2023.00028>
- [59] Teylo, L., Arantes, L., Sens, P., & Drummond, L. M. d. A. (2023). Scheduling bag-of-tasks in clouds using spot and burstable virtual machines. *IEEE Transactions on Cloud Computing*, 11(1), 984–998. <https://doi.org/10.1109/TCC.2021.3125426>
- [60] Thorpe, J., Zhao, P., Eyolfson, J., Qiao, Y., Jia, Z., Zhang, M., Netravali, R., & Xu, G. H. (2023). Bamboo: Making preemptible instances resilient for affordable training of large DNNs. *20th USENIX Symposium on Networked Systems Design and Implementation (NSDI '23)*, 497–513. <https://www.usenix.org/conference/nsdi23/presentation/thorpe>
- [61] Tomar, S., Mishra, A. K., & Yadav, D. K. (2023). Knowledge-based checkpointing strategy for spot instances in cloud computing. *Journal of Current Science and Technology*, 13(2), 412–427. <https://doi.org/10.59796/jcst.V13N2.2023.1754>
- [62] Yang, F., Wang, L., Xu, Z., Zhang, J., Li, L., Qiao, B., Couturier, C., Bansal, C., Ram, S., Qin, S., Ma, Z., Goiri, Í., Cortez, E., Yang, T., Rühle, V., Rajmohan, S., Lin, Q., & Zhang, D. (2023). Snape: Reliable and low-cost computing with mixture of spot and on-demand VMs. *Proceedings of the 28th ACM International Conference on Architectural Support for*

Programming Languages and Operating Systems, Volume 3 (ASPLOS '23).
<https://doi.org/10.1145/3582016.3582028>

- [63] Apache Software Foundation. (2024). *Apache OpenWhisk: Open source serverless cloud platform*. <https://openwhisk.apache.org/>
- [64] Basu Roy, R., & Tiwari, D. (2024). StarShip: Mitigating I/O bottlenecks in serverless computing for scientific workflows. *Abstracts of the 2024 ACM SIGMETRICS/IFIP PERFORMANCE Joint International Conference*, 79–80. <https://doi.org/10.1145/3652963.3655082>
- [65] Callaghan, S., Maechling, P. J., Goulet, C. A., Milner, K. R., Graves, R. W., Olsen, K. B., Cui, Y., Vahi, K., Rynge, M., Ferreira da Silva, R., Deelman, E., & Jordan, T. H. (2024). CyberShake: Running seismic hazard workflows on distributed high-performance computing resources. *Proceedings of the 20th IEEE International Conference on e-Science (e-Science '24)*. <https://doi.org/10.5072/par-10630254>
- [66] Chen, L., Liu, G., Zhang, J., & Zhang, X. (2024). Scheduling ensemble workflows on hybrid resources in IaaS clouds. *Computing*, 107(1). <https://doi.org/10.1007/s00607-024-01386-8>
- [67] Cho, Y., Kim, Y., Kim, K., Kim, J., Kim, H.-Y., & Kim, Y. (2024). Optimizing multi-level checkpointing for distributed deep learning workloads on cloud spot VM clusters. *IEEE Access*, 12, 116891–116904. <https://doi.org/10.1109/ACCESS.2024.3446770>
- [68] Duan, J., Song, Z., Miao, X., Xi, X., Lin, D., Xu, H., Zhang, M., & Jia, Z. (2024). Parcae: Proactive, liveput-optimized DNN training on preemptible instances. *Proceedings of the 21st USENIX Symposium on Networked Systems Design and Implementation*.
- [69] Kohli, S., Kharbanda, S., Bruno, R., Carreira, J., & Fonseca, P. (2024). Pronghorn: Effective checkpoint orchestration for serverless hot-starts. *Proceedings of the Nineteenth European Conference on Computer Systems (EuroSys '24)*, 298–316. <https://doi.org/10.1145/3627703.3629556>
- [70] Liu, F., & Niu, Y. (2024). Demystifying the cost of serverless computing: Towards a win-win deal. *IEEE Transactions on Parallel and Distributed Systems*, 35(1), 59–72. <https://doi.org/10.1109/TPDS.2023.3330849>
- [71] Miao, X., Shi, C., Duan, J., Xi, X., Lin, D., Cui, B., & Jia, Z. (2024). Spot-Serve: Serving generative large language models on preemptible instances. *Proceedings of the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems (ASPLOS '24)*, 1112–1127. <https://doi.org/10.1145/3620666.3651335>
- [72] Nag, U., Sharan, A., & Kalra, M. (2024). Cost-deadline constrained robust scheduling of workflows using hybrid instances in IaaS cloud. *Artificial*

- Intelligence of Things*, 227–240. https://doi.org/10.1007/978-3-031-48774-3_16
- [73] Son, M., Akbulut, G. G., & Kandemir, M. T. (2024). SpotVerse: Optimizing bioinformatics workflows with multi-region spot instances in Galaxy and beyond. *Proceedings of the 25th International Middleware Conference*, 74–87. <https://doi.org/10.1145/3652892.3700750>
 - [74] Triendl, S., & Ristov, S. (2024). Peeking behind the serverless implementations and deployments of the Montage workflow. *Companion of the 15th ACM/SPEC International Conference on Performance Engineering (ICPE '24)*, 196–203. <https://doi.org/10.1145/3629527.3651420>
 - [75] Wang, Y., Zuo, X., Wang, H., & Wu, Z. (2024). Real-time dynamic cloud workflow scheduling for volatile resources in clouds [Preprint available at SSRN]. <https://doi.org/10.2139/ssrn.4903286>
 - [76] Wu, Z., Chiang, W.-L., Mao, Z., Yang, Z., Friedman, E., Shenker, S., & Stoica, I. (2024). Can't be late: Optimizing spot instance savings under deadlines. *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*, 185–203. <https://www.usenix.org/conference/nsdi24/presentation/wu-zhanghao>
 - [77] Zhang, Z., Jin, C., & Jin, X. (2024). Jolteon: Unleashing the promise of serverless for serverless workflows. *21st USENIX Symposium on Networked Systems Design and Implementation (NSDI '24)*, 1469–1487. <https://www.usenix.org/conference/nsdi24/presentation/zhang-zili-jolteon>
 - [78] Duan, J., Xu, S., Qian, S., Yang, D., Wang, K., Liao, C., Yu, Y., Hua, Q., Hu, H., Wang, Q., Wu, W., Bao, D., Lu, T., Cao, J., Xue, G., Yang, G., Zhang, L., & Chen, G. (2025). GFS: A preemption-aware scheduling framework for GPU clusters with predictive spot instance management. *Proceedings of the 31st ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 1*, 117–131. <https://doi.org/10.1145/3760250.3762231>
 - [79] Mao, Z., Xia, T., Wu, Z., Chiang, W.-L., Griggs, T., Bhardwaj, R., Yang, Z., Shenker, S., & Stoica, I. (2025). SkyServe: Serving AI models across regions and clouds with spot instances. *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*, 159–175. <https://doi.org/10.1145/3689031.3717459>
 - [80] Samanta, A., Stutsman, R., & Roy, R. B. (2025). GridGreen: Integrating serverless computing in HPC systems for performance and sustainability. *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC '25)*, 388–401. <https://doi.org/10.1145/3772052.3772241>

- [81] Schmid, L., Copik, M., Calotoiu, A., Brandner, L., Koziolk, A., & Hoefler, T. (2025). SeBS-Flow: Benchmarking serverless cloud function workflows. *Proceedings of the Twentieth European Conference on Computer Systems (EuroSys '25)*. <https://doi.org/10.1145/3689031.3717465>
- [82] Sinha, A., Li, Z., Liu, T., Wang, S., Xiao, N., & Zhou, H. (2025). Fed-CostAware: Enabling cost-aware federated learning on the cloud. <https://arxiv.org/abs/2505.21727>
- [83] Tang, W., Yang, Y., & Wu, J. (2025). Metis: A non-clairvoyant, workflow-aware OS scheduler for serverless applications. *Proceedings of the 2025 ACM Symposium on Cloud Computing (SoCC '25)*, 149–162. <https://doi.org/10.1145/3772052.3772221>
- [84] Zhu, B., Zhu, Y., Chen, C., & Kong, L. (2025). Trident: A provider-oriented resource management framework for serverless computing platforms. *IEEE Transactions on Services Computing*, 18(5), 3334–3347. <https://doi.org/10.1109/TSC.2025.3603867>
- [85] Amazon Web Services. (2026). *Spot instance interruption notices*. Retrieved March 18, 2026, from <https://docs.aws.amazon.com/AWSEC2/latest/UserGuide/spot-instance-termination-notices.html>
- [86] Apache Software Foundation. (2026). *Apache airflow documentation*. Retrieved March 18, 2026, from <https://airflow.apache.org/docs/index.html>
- [87] Bhuyan, N., Bhatia, R., Kodialam, M., & Lakshman, T. V. (2026). Opportunistic scheduling for optimal spot instance savings in the cloud. <https://arxiv.org/abs/2601.12266>
- [88] Fission Project. (2026). Fission: Open source Kubernetes-native serverless framework.
- [89] Kim, K., Song, M., Kim, T., & Lee, K. (2026). Ding-dong ditch: Peeking into spot instance availability. <https://arxiv.org/abs/2604.16457>
- [90] Li, Z., Xia, T., Mao, Z., Zhou, Z., Jackson, E. J., Kerney, J., Wu, Z., Mishra, P., Xu, Y., Qiao, Y., Shenker, S., & Stoica, I. (2026). SkyNomad: On using multi-region spot instances to minimize AI batch job cost. <https://doi.org/10.48550/arXiv.2601.06520>
- [91] Temporal Technologies. (2026). *Temporal platform documentation*. Retrieved March 18, 2026, from <https://docs.temporal.io/>
- [92] The Kubernetes Authors. (2026). Kubernetes.